Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Penataan Paket dalam Truk dengan Algoritma Pencarian Lokal



Disusun oleh

Kelompok 45: FC Barcelona

Nashiruddin Akram (13524090)

Reinsen Silitonga (13524093)

Arghawisesa Dwinanda Arham (13524100)

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

TAHUN AJARAN 2026/2027

Daftar Isi

BAB 1
Deskripsi Persoalan.....1
1.1 Latar Belakang dan Tujuan..... 1
1.2 Data dan Bentuk Solusi.....1
1.3 Fungsi Objektif dan Kendala..... 1
BAB 2
Pembahasan.....2
2.1 Struktur Proyek..... 2
2.2 Komponen Dasar Program.....3
2.2.1 Representasi Data dan Orientasi..... 3
2.2.2 Validasi Kelayakan dan Fungsi Objektif.....3
2.2.3 Pemilihan Posisi dan Inisialisasi State..... 4
2.2.4 Pembentukan dan Pemilihan Neighbor.....5
2.3 Hill-Climbing..... 6
2.3.1 Steepest Ascent Hill-Climbing.....6
2.3.2 Sideways Move Hill-Climbing..... 6
2.3.3 Stochastic Hill-Climbing.....7
2.3.4 Random Restart Hill-Climbing..... 7
2.4 Simulated Annealing.....8
2.4.1 Penerimaan Kandidat..... 8
2.4.2 Jadwal Suhu..... 9
2.4.3 Pencatatan Metrik Pencarian.....9
2.5 Genetic Algorithm.....10
2.5.1 Representasi Individu dan Seleksi Induk..... 10
2.5.2 Crossover, Repair, dan Mutasi..... 10
2.5.3 Evolusi Populasi dan Elitisme.....10
2.5.4 Pencatatan Hasil Pencarian..... 11
2.6 Masukan, Keluaran, dan Visualisasi..... 12
2.6.1 Konfigurasi dan Format Masukan.....12
2.6.2 Penyimpanan Hasil Pencarian.....13
2.6.3 Grafik dan Antarmuka Visualisasi..... 13
2.7 Rancangan Eksperimen.....14
2.7.1 Kasus Uji dan Cara Membaca Visualisasi..... 14
2.7.2 Pengaturan Pengujian dan Ukuran Penilaian.....15
2.8 Pengujian Parameter Genetic Algorithm..... 16
2.8.1 Pengaruh Jumlah Generasi.....16
2.8.2 Contoh Perkembangan pada Generasi yang Berbeda..... 17
2.8.3 Pengaruh Jumlah Individu dalam Populasi.....18
2.8.4 Pengaruh Populasi Sejak State Awal.....19

2.9 Hasil Pencarian Hill-Climbing....20
2.9.1 Ringkasan Empat Varian....20
2.9.2 Contoh State pada Steepest Ascent Hill-Climbing....21
2.9.3 Contoh State pada Sideways Move Hill-Climbing....22
2.9.4 Contoh State pada Stochastic Hill-Climbing....23
2.9.5 Contoh State dan Restart pada Random Restart Hill-Climbing....24
2.10 Hasil Pencarian Simulated Annealing....25
2.10.1 Ringkasan Nilai, Waktu, dan Kondisi Pencarian....25
2.10.2 Ketika Nilai Akhir Lebih Rendah daripada Nilai Terbaik....26
2.10.3 Contoh Pencarian SA yang Meningkatkan Nilai....27
2.11 Hasil Pencarian Genetic Algorithm....28
2.11.1 Perubahan Nilai pada Enam Run....28
2.11.2 Contoh State Terbaik yang Ditemukan GA....29
2.12 Perbandingan dan Analisis Akhir....30
2.12.1 Kualitas Hasil, Waktu, dan Konsistensi....30
2.12.2 Perbandingan State Akhir dari State Awal yang Sama....32
2.12.3 Kedekatan dengan Nilai Terbaik dan Batas Kesimpulan....33
2.13 Bonus: Fungsi Objektif Berbasis ETA....34
BAB 3
Kesimpulan dan Saran....35
3.1 Kesimpulan....35
3.2 Saran....35
BAB 4
Pembagian Tugas Anggota....36
Referensi....37
Deklarasi Penggunaan AI....38

BAB 1
Deskripsi Persoalan

1.1 Latar Belakang dan Tujuan

Penataan paket di dalam truk perlu mempertimbangkan ruang, berat, dan keamanan barang. Paket yang bernilai tinggi belum tentu dapat dimasukkan apabila ukurannya tidak sesuai atau susunannya melanggar aturan. Karena kapasitas truk terbatas, program perlu memilih paket yang dibawa sekaligus menentukan posisi dan orientasinya.

Tugas besar ini bertujuan mencari susunan paket yang memaksimalkan total nilai paket di dalam truk. Penyelesaiannya menggunakan Hill-Climbing (HC), Simulated Annealing (SA), dan Genetic Algorithm (GA).

1.2 Data dan Bentuk Solusi

Masukan terdiri atas ukuran dan kapasitas berat truk, serta daftar paket. Setiap paket memiliki ID, dimensi, nilai, berat, status pecah-belah, perkiraan waktu kedatangan (*estimated time of arrival*, ETA), dan orientasi. Koordinat penempatan menggunakan bilangan bulat. Paket dapat diputar ke enam orientasi berdasarkan pemetaan sisinya terhadap sumbu *x*, *y*, dan *z*.

Satu *state* menyimpan penempatan seluruh paket. Paket di dalam truk mempunyai posisi (x_i, y_i, z_i) dan orientasi. Paket yang tidak dibawa tetap dicatat dengan posisi `None`. Dengan bentuk ini, program dapat menunjukkan paket yang masuk maupun yang berada di luar truk.

1.3 Fungsi Objektif dan Kendala

Nilai suatu State dihitung dengan menjumlahkan nilai seluruh paket yang masuk truk.

$$f(s) = \sum_{i=1}^{n} v_i u_i(s),$$

dengan v_i sebagai nilai paket dan $u_i(s) = 1$ jika paket masuk, atau bernilai 0 jika berada di luar. ETA digunakan pada bonus 1 melalui fungsi objektif kedua berikut.

$$f_{ETA}(s) = \sum_{i=1}^{n} \frac{v_i u_i(s)}{1 + ETA_i/2}$$

dengan ETA_i sebagai ETA paket ke-i. Bobot paket mengecil ketika ETA membesar sehingga paket dengan ETA kecil lebih diutamakan. Penjelasan fungsi ini terdapat pada Subbab 2.13.

State yang diterima harus memenuhi aturan berikut.

1. Paket di dalam harus seluruhnya berada dalam batas truk.
2. Paket tidak boleh saling menembus. Permukaan yang hanya bersentuhan diperbolehkan.
3. Paket harus berada di lantai atau memiliki penopang di bawah sebagian alasnya.
4. Paket pecah-belah tidak boleh menjadi penopang paket lain.
5. Berat total paket yang dibawa tidak boleh melebihi kapasitas truk.

BAB 2
Pembahasan

2.1 Struktur Proyek

Program disusun dalam beberapa bagian agar representasi persoalan, pencarian solusi, dan penyajian hasil dapat dikembangkan secara terpisah. Struktur direktori utama proyek ditunjukkan berikut.

Struktur direktori proyek.

```
IF3170_G45_FC-Barcelona/
|-- docs/
|   |-- laporan.pdf
|-- examples/
|   |-- benchmark20.json
|   |-- tiny.json
|-- output/
|-- src/
|   |-- algorithms/
|   |   |-- common.py
|   |   |-- hillclimbing.py
|   |   |-- sa.py
|   |   |-- genetic.py
|   |-- web/
|   |   |-- index.html
|   |-- model.py
|   |-- validation.py
|   |-- initialize.py
|   |-- neighbors.py
|   |-- results.py
|   |-- visualization.py
|   |-- web.py
|-- .gitignore
|-- README.md
|-- requirements.txt
|-- requirements-lock.txt
```

Folder `src` memuat implementasi program. Algoritma menggunakan model data, validasi, dan pembentukan *neighbor* yang sama. Folder `examples` menyimpan masukan persoalan dan `output` menampung hasil pencarian. Folder docs menyimpan laporan.pdf, sedangkan README.md berada di root repositori.

2.2 Komponen Dasar Program

HC, SA, dan GA memakai model data, validasi, inisialisasi, dan pembentukan *neighbor* yang sama. Komponen ini memastikan setiap *state* yang dinilai memenuhi aturan persoalan.

2.2.1 Representasi Data dan Orientasi

Model data berada pada `model.py`. Pemisahan objek persoalan dan *state* pencarian memungkinkan satu masukan digunakan oleh beberapa algoritma tanpa mengubah atribut asli paket.

Kelas	Peran dalam program
`Truck`	Menyimpan dimensi truk dan kapasitas berat.
`Package`	Menyimpan ID, dimensi, nilai, berat, status pecah-belah, ETA, dan orientasi.
`Problem`	Menggabungkan truk dan daftar paket dengan ID yang unik.
`Placement`	Menyimpan posisi dan orientasi satu paket.
`State`	Memetakan seluruh ID paket ke penempatannya.

Tabel 2.1: Kelas utama untuk merepresentasikan persoalan.

Pemetaan penempatan disalin saat membentuk *state*. Perubahan pada data asal tidak mengubah *state* tersebut. Saat menghasilkan *neighbor*, program membuat *state* baru dari salinan penempatan.

Kode 2.1: Penyimpanan salinan penempatan pada kelas `State`.

```
@dataclass(frozen=True)
class State:
    placements: Mapping[str, Placement]
    def __post_init__(self) -> None:
        placements = dict(self.placements)
        if any(not isinstance(key, str) or not key for key in placements):
            raise ValueError("state.placements harus memakai ID string tidak
kosong")
        if any(not isinstance(item, Placement) for item in placements.values()):
            raise ValueError("state.placements harus berisi Placement")
        object.__setattr__(self, "placements", MappingProxyType(placements))
```

Enam orientasi paket adalah `xyz`, `xzy`, `yxz`, `yzx`, `zxy`, dan `zyx`. Fungsi `oriented_size` menghitung ukuran pada tiap sumbu. Fungsi `rotate` menukar dua sumbu yang tegak lurus terhadap sumbu rotasi.

Kode 2.2: Penghitungan dimensi berdasarkan orientasi dan rotasi.

```
def oriented_size(package: Package, orientation: Orientation) -> Position:
    axes = dict(zip(orientation.value, package.dimensions))
    return axes["x"], axes["y"], axes["z"]

def rotate(orientation: Orientation, axis: str) -> Orientation:
    if axis not in ("x", "y", "z"):
        raise ValueError("axis harus x, y, atau z")
    a, b = [item for item in "xyz" if item != axis]
    return Orientation(orientation.value.translate(str.maketrans({a: b, b: a})))
```

2.2.2 Validasi Kelayakan dan Fungsi Objektif

Fungsi `validate` memeriksa kesesuaian ID paket, batas ruang, berat, irisan antarpaket, dan penopang. Batas ruang dihitung dari posisi dan dimensi setelah orientasi. Total berat dijumlahkan dengan `fsum` dan dibandingkan dengan kapasitas truk menggunakan toleransi numerik `isclose`.

Kode 2.3: Pemeriksaan batas ruang, berat, dan irisan paket.

```
weight = []
for package in problem.packages:
    placement = state.placements[package.id]
    if placement.position is None:
        continue
    lower = placement.position
    upper = tuple(a + b for a, b in zip(lower, oriented_size(package,
placement.orientation)))
    if any(a < 0 or b > limit for a, b, limit in zip(lower, upper,
problem.truck.dimensions)):
        reasons.append(f"{package.id}: menembus dinding")
    boxes[package.id] = (lower, upper, package.is_fragile)
    weight.append(package.weight)
total = fsum(weight)
capacity = problem.truck.max_capacity
if total > capacity and not isclose(total, capacity, rel_tol=1e-12, abs_tol=1e-12):
    reasons.append(f"Berat {total:g} melebihi kapasitas {capacity:g}")
for (a_id, (a, aa, _)), (b_id, (b, bb, _)) in combinations(boxes.items(), 2):
    if all(max(a[i], b[i]) < min(aa[i], bb[i]) for i in range(3)):
        reasons.append(f"{a_id} dan {b_id}: overlap")
```

Paket di lantai langsung dianggap memiliki penopang. Untuk paket di atas lantai, permukaan atas penopang harus tepat berada pada ketinggian alas dan beririsan pada bidang *x*–*y*. Penopang tidak boleh berupa paket pecah-belah.

Kode 2.4: Pemeriksaan penopang dan status pecah-belah.

```
for package_id, (lower, upper, _) in boxes.items():
    if lower[2] == 0:
        continue
    supporters = []
    for other_id, (other, top, fragile) in boxes.items():
        if other_id == package_id or top[2] != lower[2]:
            continue
        if all(max(lower[i], other[i]) < min(upper[i], top[i]) for i in (0, 1)):
            supporters.append(other_id)
            if fragile:
                reasons.append(f"{package_id}: ditopang paket fragile {other_id}")
    if not supporters:
        reasons.append(f"{package_id}: melayang")
return ValidationResult(not reasons, tuple(reasons))
```

Paket di luar truk tidak menyumbang berat atau nilai. Fungsi `objective` memanggil `require_valid` sebelum menjumlahkan nilai paket yang masuk. Kandidat tidak valid ditolak, sedangkan fungsi objektif hanya dipakai untuk menilai *state* yang memenuhi kendala.

Kode 2.5: Penghitungan nilai *state* yang valid.

```
def objective(problem: Problem, state: State) -> Number:
    require_valid(problem, state)
    return fsum(p.value for p in problem.packages if state.placements[p.id].position
is not None)
```

2.2.3 Pemilihan Posisi dan Inisialisasi *State*

Fungsi `candidate_position` memilih koordinat berdasarkan ruang yang tersedia untuk ukuran paket setelah orientasi. Batas tiap sumbu adalah ukuran truk dikurangi ukuran paket. Kandidat *x* dan *y* dipilih dari koordinat bulat dalam batas tersebut. Kandidat *z* berasal dari lantai atau ketinggian permukaan atas paket yang tidak mudah pecah yang telah ditempatkan.

Kode 2.6: Pemilihan calon koordinat paket.

```
def candidate_position(problem, state, package, orientation, rng):
    size = oriented_size(package, orientation)
    limits = tuple(a - b for a, b in zip(problem.truck.dimensions, size))
    if min(limits) < 0:
        return None
    levels = {0}
    for other in problem.packages:
        place = state.placements[other.id]
        if place.position is not None and not other.is_fragile:
            top = place.position[2] + oriented_size(other, place.orientation)[2]
            if top <= limits[2]:
                levels.add(top)
    return rng.randrange(limits[0] + 1), rng.randrange(limits[1] + 1),
rng.choice(sorted(levels))
```

Fungsi `random_feasible_state` memulai pencarian dengan seluruh paket di luar. Urutan paket diacak, lalu posisi dan orientasinya dicoba satu per satu. Penempatan disimpan hanya jika seluruh *state* valid. Paket yang gagal ditempatkan dalam batas percobaan tetap berada di luar.

Kode 2.7: Pembentukan *state* awal yang memenuhi kendala.

```
def random_feasible_state(problem: Problem, rng: Random, max_attempts: int = 100) ->
State:
    if isinstance(max_attempts, bool) or not isinstance(max_attempts, int) or
max_attempts <= 0:
        raise ValueError("max_attempts harus integer positif")
    state = State.empty(problem)
    packages = list(problem.packages)
    rng.shuffle(packages)
    for package in packages:
        for _ in range(max_attempts):
            orientation = rng.choice(list(Orientation))
            position = candidate_position(problem, state, package, orientation, rng)
            if position is None:
                continue
            placements = dict(state.placements)
            placements[package.id] = Placement(position, orientation)
            candidate = State(placements)
            if validate(problem, candidate).valid:
                state = candidate
                break
    return state
```

Pemilihan ketinggian calon penopang mengurangi percobaan pada posisi yang jelas melayang. Validasi tetap diperlukan karena koordinat x dan y yang terpilih belum tentu berada tepat di atas penopang, dan kandidat masih dapat melanggar batas berat atau menabrak paket lain.

2.2.4 Pembentukan dan Pemilihan *Neighbor*

Modul `neighbors.py` menyediakan *move* yang digunakan selama pencarian. Jenis *move* dinyatakan oleh `MoveKind`, sedangkan `Move` menyimpan ID paket dan rincian perubahan. `NeighborResult` mencatat kandidat, *move* yang dipakai, jumlah percobaan, dan alasan kegagalan memperoleh *neighbor*. Fungsi `apply_move` menyalin penempatan dan menerapkan salah satu dari tiga *move*. *Swap* hanya menukar posisi, sehingga orientasi tetap mengikuti masing-masing paket. *Relocate* mengganti koordinat satu paket, termasuk `None` untuk mengeluarkannya. *Rotate* mengubah orientasi dengan mempertahankan titik awal paket.

Kode 2.8: Penerapan *move* dan validasi kandidat.

```
def apply_move(problem, state, move):
    require_valid(problem, state)
    if move.package_id not in state.placements:
        raise ValueError("ID move tidak dikenal")
    placements = dict(state.placements)
    first = placements[move.package_id]
    if move.kind == MoveKind.SWAP:
        if move.other_id not in placements:
            raise ValueError("ID pasangan swap tidak dikenal")
        second = placements[move.other_id]
        placements[move.package_id] = Placement(second.position, first.orientation)
        placements[move.other_id] = Placement(first.position, second.orientation)
    elif move.kind == MoveKind.RELOCATE:
        placements[move.package_id] = Placement(move.position, first.orientation)
    elif move.kind == MoveKind.ROTATE:
        placements[move.package_id] = Placement(first.position, rotate(first.orientation,
move.axis))
    else:
        raise ValueError("Jenis move tidak dikenal")
    candidate = State(placements)
    if candidate == state or not validate(problem, candidate).valid:
        return None
    return candidate
```

Fungsi `sample_feasible_neighbor` memilih jenis *move* dan paket secara acak, lalu mencoba sampai menemukan kandidat valid atau mencapai batas percobaan. Pada *relocate* untuk paket di dalam, terdapat peluang 0,2 untuk mengusulkan pengeluaran paket. Aturan penerimaan algoritma tetap menentukan apakah kandidat itu dipakai. Untuk pencarian menyeluruh, `iter_feasible_neighbors` menghasilkan kandidat *swap*, *rotate*, dan *relocate*. Fungsi `best_feasible_neighbor` mengevaluasinya satu per satu dan menyimpan nilai tertinggi. Jika beberapa kandidat terbaik memiliki nilai sama, salah satunya dipilih secara acak.

2.3 Hill-Climbing

Hill-Climbing memperbaiki satu State secara bertahap dengan berpindah ke neighbor yang nilainya lebih tinggi. Pada penataan paket, perubahan dilakukan pada susunan yang sedang dicoba untuk meningkatkan total nilai muatan. Pencarian dapat terhenti ketika tidak ada perbaikan di sekitarnya, meskipun solusi yang lebih baik mungkin ada di bagian lain ruang pencarian. Empat varian yang digunakan membedakan cara memilih tetangga dan melanjutkan pencarian.

2.3.1 Steepest Ascent Hill-Climbing

Fungsi `steepest_ascent` memilih *neighbor* dengan nilai tertinggi pada setiap iterasi. Kandidat diterima jika nilainya lebih tinggi daripada *state* sekarang. Pencarian berhenti ketika tidak ada *neighbor*, tidak ada perbaikan, atau batas iterasi tercapai.

Kode 2.9: Pemilihan *move* pada Steepest Ascent Hill-Climbing.

```
for _ in range(config["iterations"]):
    neighbor, neighbor_score, evaluated = best_feasible_neighbor(problem, current,
rng)
    evaluated_neighbors += evaluated
    if neighbor is None or neighbor_score <= score:
        history.append(score)
        stop_reason = "no_feasible_neighbor" if neighbor is None else
"local_optimum"
        break
    current, score = neighbor, neighbor_score
    accepted_moves += 1
    history.append(score)
```

2.3.2 Sideways Move Hill-Climbing

Fungsi `sideways` mengizinkan *sideways move*, yaitu perpindahan ke *neighbor* yang nilainya sama, untuk mengubah posisi sebelum ada peningkatan. Jumlah *sideways move* dihitung secara total dan tidak direset ketika nilai naik.

Kode 2.10: Penerimaan *sideways move* dan batas totalnya.

```
for _ in range(config["iterations"]):
    neighbor, neighbor_score, evaluated = best_feasible_neighbor(problem, current,
rng)
    evaluated_neighbors += evaluated
    if neighbor is None or neighbor_score < score:
        history.append(score)
        stop_reason = "no_feasible_neighbor" if neighbor is None else
"local_optimum"
        break
    if neighbor_score == score:
        if config["max_sideways_moves"] == 0:
            history.append(score)
            stop_reason = "sideways_limit"
            break
        sideways_moves += 1
    current, score = neighbor, neighbor_score
    accepted_moves += 1
    history.append(score)
    if sideways_moves > 0 and sideways_moves >= config["max_sideways_moves"]:
        stop_reason = "sideways_limit"
        break
```

Jika `max_sideways_moves` bernilai nol, perbaikan masih boleh diterima tetapi *sideways move* ditolak. Untuk batas positif, pencarian berhenti segera setelah jumlah *sideways move* mencapai batas. Metrik menyimpan total *sideways move* dan jumlah *move* yang diterima.

2.3.3 Stochastic Hill-Climbing

Fungsi `stochastic` mengambil satu *neighbor* valid secara acak pada setiap iterasi. Kandidat diterima hanya jika nilainya lebih tinggi. Kandidat yang nilainya sama atau lebih buruk ditolak, sedangkan kegagalan mengambil kandidat membuat program mencoba lagi pada iterasi berikutnya.

Kode 2.11: Pengambilan dan penerimaan *neighbor* pada Stochastic Hill-Climbing.

```
for _ in range(config["iterations"]):
    sampled = sample_feasible_neighbor(problem, current, rng,
config["max_attempts"])
    sampling_attempts += sampled.attempts
    if sampled.state is None:
        sampling_failures += 1
        history.append(score)
        continue
    neighbor_score = objective(problem, sampled.state)
    evaluated_neighbors += 1
    if neighbor_score > score:
        current, score = sampled.state, neighbor_score
        accepted_moves += 1
    history.append(score)
```

Implementasi Stochastic Hill-Climbing berhenti pada batas iterasi. Batas iterasi ini tidak membuktikan bahwa seluruh *neighbor* sudah tidak dapat memperbaiki *state*. Riwayat nilai tetap dicatat pada iterasi yang tidak menghasilkan *move*.

2.3.4 Random Restart Hill-Climbing

Fungsi `random_restart` menjalankan Steepest Ascent Hill-Climbing berulang kali dari *state* awal yang berbeda. Hasil terbaik seluruh pencarian dipertahankan, sedangkan hasil tiap pencarian disimpan untuk melihat jumlah iterasi dan alasan berhenti.

Kode 2.12: *Restart* pencarian dari beberapa *state* awal.

```
for restart in range(config["max_restarts"] + 1):
    start = first if restart == 0 else initial_state(problem, rng, None)
    result = steepest_ascent(problem, base_config, start)
    if result.final_score > best_score:
        best, best_score = result.final_state, result.final_score
    history.extend(result.objective_history)
    runs.append(dict(restart=restart, initial_state=start.to_dict(),
                     final_state=result.final_state.to_dict(),
                     objective_history=list(result.objective_history),
                     stop_reason=result.stop_reason, **result.metrics))
```

Nilai `max_restarts=3` menghasilkan satu pencarian awal dan tiga *restart*. Riwayat tiap pencarian dicatat terpisah karena nilai awal setelah *restart* dapat lebih rendah daripada hasil pencarian sebelumnya. Hasil akhir yang dikembalikan adalah *state* terbaik dari seluruh pencarian tersebut.

2.4 Simulated Annealing

Simulated Annealing menelusuri neighbor secara acak dan dapat menerima solusi yang nilainya lebih rendah untuk membuka peluang keluar dari optimum lokal. Pada suhu tinggi, pencarian lebih mudah menerima penurunan nilai. Suhu kemudian diturunkan secara bertahap sehingga pencarian semakin selektif dan lebih berfokus pada perbaikan solusi.

2.4.1 Penerimaan Kandidat

Fungsi `simulated_annealing` menghitung selisih $\Delta = f(s') - f(s)$. Kandidat yang meningkatkan nilai langsung diterima. Untuk kandidat lain, `acceptance_probability` menghitung peluang penerimaan dengan rumus berikut.

$$p(\Delta, T) = min(1, exp(\Delta/T)),\ T > 0$$

Program mengambil bilangan acak dari nol sampai kurang dari satu. Kandidat diterima jika bilangan tersebut lebih kecil daripada peluang penerimaannya. Kandidat dengan nilai yang sama selalu diterima, sedangkan peluang menerima kandidat yang lebih buruk bergantung pada suhu dan besar penurunan nilainya.

Kode 2.13: Evaluasi kandidat dan pembaruan *state* pada SA.

```
next_state = samples.state
next_score = objective(problem, next_state)
delta = next_score - score
if delta > 0:
    accepted = True
    probability = 1.0
else:
    probability = acceptance_probability(delta, temperature)
    accepted = randomm.random() < probability
previous_stuck = stuck
if accepted:
    current = next_state
    score = next_score
    accepted_moves += 1
    if delta < 0:
        accepted_moves_worse += 1
    if score > best_score:
        best = current
        best_score = score
    stuck = not improving_neighbor_exists(problem, current)
```

SA menyimpan *State* terakhir dan *State* terbaik secara terpisah. *State* terakhir mengikuti kandidat yang diterima, sedangkan *State* terbaik diperbarui hanya ketika pencarian menemukan nilai yang lebih tinggi daripada semua nilai sebelumnya.

2.4.2 Jadwal Suhu

Suhu diatur oleh temperature_schedule menggunakan penurunan geometrik berikut.

$$T_t = T_0 \alpha^{t-1}, \qquad 0 < \alpha \leq 1.$$

Untuk $\alpha < 1$, peluang menerima penurunan nilai mengecil seiring turunnya suhu. Nilai $\alpha = 1$ mempertahankan suhu sampai batas iterasi. Pencarian dihentikan ketika suhu mencapai batas minimum atau batas iterasi tercapai.

Kode 2.14: Jadwal suhu geometrik pada SA.

```
def temperature_schedule(t, config):
    if t > config["iterations"]:
        return 0.0
    temperature = (config["initial_temperature"] * config["cooling_rate"] ** (t-1))
    if temperature <= config["min_temperature"]:
        return 0.0
    return temperature
```

2.4.3 Pencatatan Metrik Pencarian

Fungsi `improving_neighbor_exists` mencari perbaikan melalui penambahan paket dari luar atau *swap* antara paket luar yang nilainya lebih tinggi dan paket di dalam. Move lain tidak meningkatkan total nilai secara langsung. Pemeriksaan berhenti begitu menemukan kandidat valid yang memperbaiki nilai. Jika tidak ditemukan, State dicatat sebagai stuck. Kondisi ini juga mencakup plateau dan optimum global.

Program mencatat jumlah periode dan iterasi *stuck*. State awal yang sudah stuck dihitung sebagai satu periode. Periode berikutnya dimulai ketika pencarian kembali memasuki kondisi tersebut. Persentase *stuck* dihitung dari jumlah iterasi *stuck* dibagi jumlah iterasi yang selesai, tanpa menghitung *state* awal sebagai iterasi. Catatan tiap iterasi memuat suhu, selisih nilai, dan keputusan penerimaan.

Kode 2.15: Pencatatan perkembangan nilai dan metrik SA.

```
history.append(score)
stuck_history.append(stuck)
records.append({"iteration": t,
                "temperature": temperature,
                "delta": delta,
                "accepted": accepted,
                "objective": score,
                "acceptance_probability": probability,
                "stuck": stuck,
                **_exponential_metrics(delta, temperature),})
t += 1
```

Fungsi `_exponential_metrics` mencatat hasil rumus eksponensial sebelum dibatasi menjadi peluang beserta penanda jika nilainya melampaui rentang bilangan. Rasio ini dibedakan dari peluang penerimaan karena dapat melebihi satu pada kandidat yang lebih baik. Grafik memberi batas tampilan pada 10^6 tanpa mengubah keputusan penerimaan algoritma.

2.5 Genetic Algorithm

Genetic Algorithm mencari solusi dengan mengembangkan sekumpulan kandidat yang disebut populasi. Kandidat yang lebih baik memiliki peluang lebih besar untuk dipilih sebagai induk. Bagian solusi dari induk digabungkan melalui crossover, lalu mutasi memberi variasi pada keturunannya. Proses ini diulang selama beberapa generasi agar populasi menghasilkan susunan paket dengan nilai muatan yang lebih tinggi. Sebagian kandidat terbaik dipertahankan melalui elitisme.

2.5.1 Representasi Individu dan Seleksi Induk

Satu individu GA adalah satu `State` lengkap. Fungsi `roulette` memilih induk dengan peluang sebanding dengan nilainya. Individu bernilai lebih tinggi lebih mungkin terpilih. Jika seluruh nilai nol, induk dipilih secara acak.

Kode 2.16: Pemilihan induk menggunakan *roulette wheel selection*.

```
def roulette(population, scores, rng):
    total = fsum(scores)
    if total <= 0:
        return rng.choice(population)
    point, cumulative = rng.random() * total, 0.0
    for state, score in zip(population, scores):
        cumulative += score
        if point < cumulative:
            return state
    return population[-1]
```

2.5.2 *Crossover*, *Repair*, dan Mutasi

Fungsi `crossover` memilih satu titik potong pada urutan paket. Penempatan sebelum titik potong diambil dari induk pertama, sedangkan sisanya dari induk kedua. Pertukaran peran induk menghasilkan dua anak.

Kode 2.17: Penggabungan penempatan paket melalui *crossover*.

```
def crossover(problem, first, second, rng):
    ids = []
    for package in problem.packages:
        ids.append(package.id)
    cut = 0
    if len(ids) > 1:
        cut = rng.randrange(1, len(ids))
    children = []
    for a, b in ((first, second), (second, first)):
        placements = {}
        for index, package_id in enumerate(ids):
            if index < cut:
                placements[package_id] = a.placements[package_id]
            else:
                placements[package_id] = b.placements[package_id]
        children.append(State(placements))
    return children
```

Anak diperiksa melalui `repair` karena hasil *crossover* belum tentu valid. Paket dimasukkan dari bawah ke atas, sedangkan penempatan yang melanggar aturan dibiarkan di luar. Fungsi `mutate` memberi variasi melalui satu move ke neighbor valid, dengan peluang yang diatur oleh parameter mutasi.

2.5.3 Evolusi Populasi dan Elitisme

Fungsi `genetic_algorithm` membentuk populasi awal dan menghitung nilai tiap individu. Pada konfigurasi eksperimen, satu individu terbaik disalin tanpa perubahan ke generasi berikutnya sebagai elit. Sisa populasi diisi melalui seleksi, *crossover*, *repair*, dan mutasi.

Kode 2.18: Pembentukan dan evaluasi populasi berikutnya pada GA.

```
following = []
left = list(range(size))
for _ in range(config["elitism"]):
    elite = left[0]
    for index in left:
        if scores[index] > scores[elite]:
            elite = index
    following.append(population[elite])
    left.remove(elite)
while len(following) < size:
    parent1 = roulette(population, scores, rng)
    parent2 = roulette(population, scores, rng)
    for child in crossover(problem, parent1, parent2, rng):
        child = repair(problem, child, rng)
        if rng.random() < config["mutation_rate"]:
            child = mutate(problem, child, rng, config["max_attempts"])
        following.append(child)
population = following[:size]
scores = []
for state in population:
    scores.append(objective(problem, state))
top = scores.index(max(scores))
if scores[top] > best_score:
    best, best_score = population[top], scores[top]
history.append(scores[top])
means.append(fsum(scores) / size)
```

Ukuran populasi tetap mengikuti nilai parameter yang dipilih. Jika pembentukan dua anak menghasilkan individu berlebih, daftar dipotong sampai ukuran populasi yang ditentukan. Setelah itu, nilai seluruh individu dihitung kembali untuk menentukan maksimum dan rata-rata generasi tersebut.

2.5.4 Pencatatan Hasil Pencarian

Program mencatat populasi awal, populasi akhir, dan *state* terbaik yang pernah ditemukan. *State* awal dan akhir yang ditampilkan mewakili individu terbaik pada populasi terkait. Populasi lengkap tetap tersedia di dalam metrik hasil pencarian. Riwayat maksimum dan rata-rata mencakup generasi awal serta setiap generasi hasil evolusi. Dengan memasukkan generasi awal, G generasi menghasilkan $G + 1$ titik pada grafik. Data ini menunjukkan perkembangan kualitas populasi. Keragaman populasi tidak dapat diukur dari maksimum dan rata-rata saja. Pengukurannya memerlukan distribusi nilai atau jumlah individu unik.

2.6 Masukan, Keluaran, dan Visualisasi

2.6.1 Konfigurasi dan Format Masukan

Fungsi `configuration` menggabungkan parameter pengguna dengan nilai bawaan, kemudian memeriksa tipe dan rentangnya. HC memakai batas 500 iterasi. SA memakai 500 iterasi, suhu awal 20, laju pendinginan 0,99, dan suhu minimum 10^{-8}. Nilai bawaan GA pada antarmuka baris perintah adalah populasi 20, 40 generasi, peluang mutasi 0,3, dan satu individu elit, sedangkan konfigurasi acuan eksperimen memakai 20 individu dan 400 generasi. Nilai ini dapat diubah melalui perintah di terminal atau antarmuka *web*. Masukan menggunakan JSON dengan objek `truck` dan daftar `packages`. Kode 2.19 menunjukkan format satu paket dengan orientasi berbentuk teks.

Kode 2.19: Contoh format masukan JSON.

```
{
  "truck": {
    "dimensions": {"w": 3, "l": 2, "h": 2},
    "maxCapacity": 5
  },
  "packages": [
    {
      "id": "A",
      "dimensions": {"w": 2, "l": 1, "h": 1},
      "orientation": "xyz",
      "value": 10,
      "weight": 2,
      "isFragile": false,
      "ETA": 1
    }
  ]
}
```

ID paket harus unik dan tidak kosong. Dimensi harus berupa bilangan bulat positif. Nilai, berat, dan kapasitas harus berupa angka nol atau positif. Nilai tak hingga tidak diperbolehkan. Status `isFragile` harus berupa *boolean* dan `ETA` berupa bilangan bulat tidak negatif. Penamaan atribut mengikuti huruf kapital dan huruf kecil pada contoh. Setiap algoritma memakai `Random` dari modul `random` Python. *Seed* mengatur urutan pilihan acak sehingga suatu pencarian dapat diulang pada masukan, pengaturan, kode, dan lingkungan yang sama. *Seed* berbeda dapat dipakai untuk mengulang eksperimen agar proses pencarian bervariasi.

2.6.2 Penyimpanan Hasil Pencarian

Hasil pencarian dikemas dalam `RunResult` agar ketiga algoritma memiliki bentuk keluaran yang konsisten. Data mencakup *state*, nilai, *seed*, konfigurasi, riwayat nilai, durasi, alasan berhenti, dan metrik khusus algoritma.

Kode 2.20: Struktur hasil pencarian pada kelas `RunResult`.

```
@dataclass(frozen=True)
class RunResult:
    algorithm: str
    seed: int
    config: Mapping[str, Any]
    initial_state: State
    final_state: State
    initial_score: Number
    final_score: Number
    objective_history: tuple[Number, ...]
    duration_seconds: float
    stop_reason: str
    best_state: State | None = None
    best_score: Number | None = None
    metrics: Mapping[str, Any] = field(default_factory=dict)
```

Fungsi check_result memeriksa kecocokan nilai terhadap State dan konsistensi riwayat serta metrik. Pembandingan suhu menggunakan toleransi relatif untuk mengakomodasi selisih pembulatan bilangan desimal. `to_document` menambahkan masukan persoalan, identitas masukan, lingkungan eksekusi, dan pemetaan sel truk. Hasil kemudian ditulis ke JSON melalui `save_result`.

Kode 2.21: Penyimpanan hasil *run* ke JSON.

```
def save_result(path, problem, result, case_name):
    document = to_document(problem, result, case_name)
    Path(path).write_text(json.dumps(document, indent=2, allow_nan=False) + "\n",
encoding="utf-8")
```

2.6.3 Grafik dan Antarmuka Visualisasi

Fungsi `plot_curves` membuat grafik nilai terhadap iterasi. Setiap pencarian pada Random Restart Hill-Climbing digambarkan terpisah. SA menambahkan grafik hasil rumus eksponensial, sedangkan GA menampilkan maksimum dan rata-rata populasi. Pengukuran durasi mencakup inisialisasi dan pencarian, tetapi tidak memasukkan penyimpanan berkas maupun pembuatan grafik. Antarmuka *web* menampilkan *state* tiga dimensi dan paket di luar truk. Pengguna dapat memilih *state* awal, akhir, atau terbaik serta mengekspor tampilan dan tabel hasil. Nama folder keluaran perlu dibedakan ketika menjalankan ulang kasus atau konfigurasi yang sama agar data sebelumnya tetap tersedia.

2.7 Rancangan Eksperimen

2.7.1 Kasus Uji dan Cara Membaca Visualisasi

Pengujian memakai satu kasus bernama `benchmark20`. Truk berukuran $10 \times 20 \times 5$ dengan kapasitas 65 unit berat. Dua puluh paket pada Tabel 2.2 memiliki ukuran, nilai, berat, dan sifat yang berbeda. Jumlah nilai seluruh paket adalah 565 dan jumlah beratnya 109. Karena berat seluruh paket melebihi kapasitas truk, program perlu memilih sebagian paket yang dapat dibawa.

Paket	Ukuran	Nilai	Berat	Pecah-belah	ETA
P01	1 × 6 × 4	49	7	Tidak	2
P02	3 × 10 × 2	25	8	Ya	1
P03	1 × 5 × 1	20	3	Tidak	1
P04	1 × 5 × 3	40	2	Tidak	0
P05	4 × 8 × 1	37	7	Tidak	3
P06	2 × 8 × 1	17	10	Tidak	0
P07	5 × 6 × 1	14	2	Ya	2
P08	2 × 10 × 3	21	9	Tidak	2
P09	6 × 7 × 4	42	10	Tidak	0
P10	6 × 3 × 4	26	1	Tidak	2
P11	5 × 10 × 1	49	2	Ya	1
P12	3 × 4 × 3	26	2	Tidak	0
P13	6 × 3 × 1	16	9	Tidak	1
P14	4 × 6 × 2	7	2	Ya	0
P15	5 × 7 × 1	26	4	Tidak	0
P16	2 × 10 × 4	44	8	Ya	0
P17	6 × 4 × 3	44	1	Ya	1
P18	5 × 8 × 1	5	9	Ya	1
P19	2 × 9 × 2	46	6	Tidak	2
P20	2 × 4 × 2	11	7	Tidak	3

Tabel 2.2: Data 20 paket pada kasus pengujian.

Terdapat tujuh paket pecah-belah. Paket tersebut boleh dibawa, tetapi tidak dapat digunakan untuk menopang paket lain. Data ETA tetap disimpan pada masukan dan tidak memengaruhi nilai State pada eksperimen fungsi objektif dasar.

Pada tampilan tiga dimensi, kerangka menunjukkan ruang truk. Paket yang ditampilkan di luar kerangka adalah barang yang tidak dibawa. Pada tampilan atas, bagian kiri menunjukkan ruang truk dan bagian kanan menunjukkan paket di luar truk. Warna mengikuti visualisasi program dan dapat dipakai berulang pada beberapa paket. Identitas setiap paket tetap mengacu pada data penempatannya.

2.7.2 Pengaturan Pengujian dan Ukuran Penilaian

Setiap konfigurasi diuji menggunakan *seed* 12, 30, 45, 66, 73, dan 86. *Seed* mengatur urutan pilihan acak, sehingga hasilnya dapat ditelusuri dan diulang. Data truk dan paket tidak berubah ketika *seed* diganti. Dengan 16 konfigurasi dan 6 *run* pada setiap konfigurasi, terdapat 96 hasil pencarian.

Bagian yang diuji	Pengaturan
Steepest Ascent Hill-Climbing	Batas 500 iterasi.
Sideways Move Hill-Climbing	Batas 500 iterasi dan total 20 *sideways move*.
Stochastic Hill-Climbing	Batas 500 iterasi.
Random Restart Hill-Climbing	Satu pencarian awal dan tiga *restart* dari *state* awal baru. Batas tiap pencarian 500 iterasi.
Simulated Annealing	500 iterasi, suhu awal 20, laju pendinginan 0,99, dan suhu minimum 10^{-8}.
GA, variasi generasi	Populasi tetap 20 individu. Jumlah generasi diubah menjadi 100, 200, 400, 600, 800, dan 1.000.
GA, variasi populasi	Jumlah generasi tetap 400. Ukuran populasi diubah menjadi 5, 10, 20, 40, 60, dan 80 individu.

Tabel 2.3: Pengaturan algoritma pada eksperimen.

GA memakai peluang mutasi 0,3 dan mempertahankan satu individu terbaik pada setiap generasi. Kedua pengujian parameter mempunyai satu konfigurasi yang sama, yaitu 20 individu dan 400 generasi. Karena itu, jumlah konfigurasi GA yang berbeda adalah 11. Batas percobaan pengambilan *neighbor* adalah 100.

Hasil dicatat pada Python 3.12.3 di Linux x86_64. Durasi pencarian mencakup inisialisasi, pencarian solusi, dan pemeriksaan yang dilakukan selama pencarian. Penyimpanan berkas serta pembuatan grafik dicatat di luar durasi tersebut. Rata-rata menunjukkan kecenderungan hasil enam *run*. Simpangan baku menunjukkan seberapa jauh hasil-hasil tersebut menyebar dari rata-ratanya. Nilai yang lebih kecil berarti hasil keenam *run* lebih berdekatan.

Hasil Sideways Move Hill-Climbing berasal dari implementasi yang dijelaskan pada Subbab 2.3.2, yaitu batas total 20 *sideways move*. Pada keenam *seed*, pencarian berhenti tepat setelah *sideways move* ke-20.

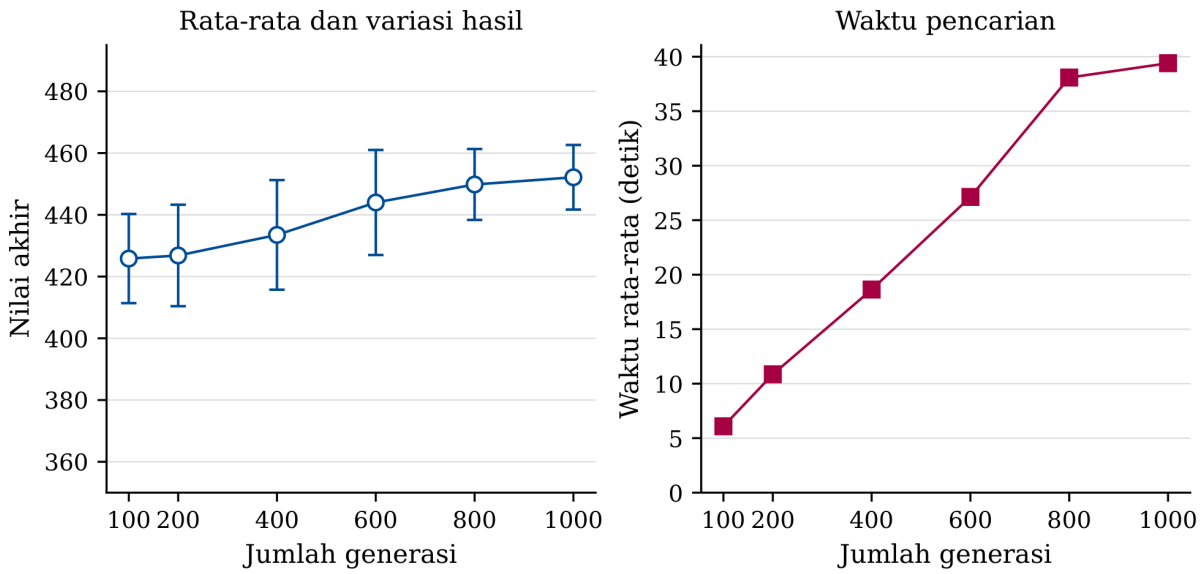
2.8 Pengujian Parameter Genetic Algorithm

2.8.1 Pengaruh Jumlah Generasi

Ukuran populasi ditahan pada 20. Enam konfigurasi pada Tabel 2.4 memakai *seed* dan parameter mutasi yang sama. Nilai awal GA adalah nilai individu terbaik pada populasi awal. Untuk satu *seed*, populasi awal tetap sama saat jumlah generasi diubah.

Generasi	Rata-rata	Simp. baku	Min–maks	Durasi rata-rata (detik)
100	425,83	14,44	412–449	6,09
200	426,83	16,44	412–455	10,86
400	433,50	17,76	414–455	18,64
600	444,00	17,02	425–467	27,13
800	449,83	11,48	435–467	38,09
1.000	452,17	10,46	435–467	39,41

Tabel 2.4: Hasil GA pada populasi 20. Setiap baris merangkum enam *run*.



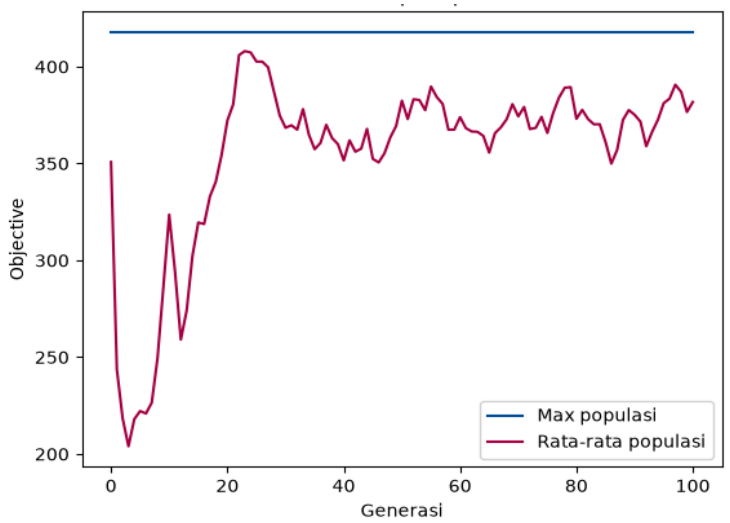
Gambar 2.1: Pengaruh jumlah generasi terhadap nilai akhir dan durasi. Batang galat menunjukkan simpangan baku sampel, bukan interval kepercayaan.

Rata-rata nilai akhir meningkat dari 425,83 pada 100 generasi menjadi 452,17 pada 1.000 generasi. Kenaikannya mencapai 26,33 atau 6,18% (dihitung dari nilai sebelum pembulatan), sedangkan durasi rata-rata bertambah dari 6,09 menjadi 39,41 detik. Penambahan generasi memberi lebih banyak kesempatan memperbaiki populasi, dengan biaya pencarian yang lebih besar.

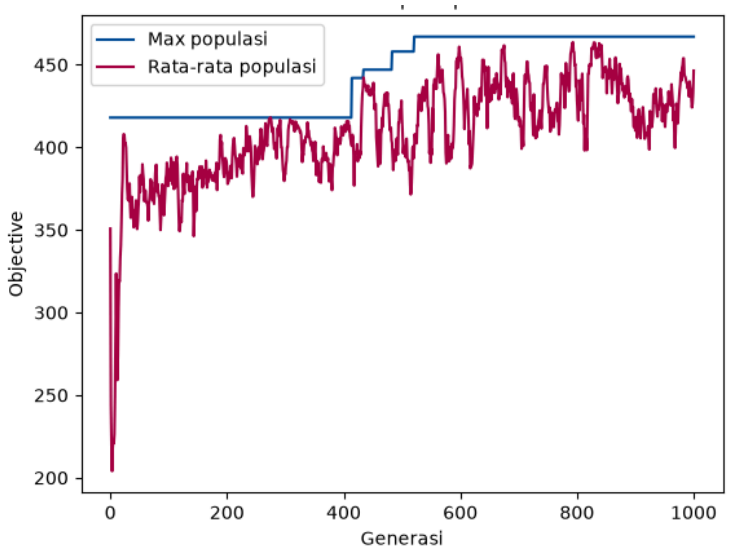
Dengan *seed* dan parameter lain yang sama, pencarian 1.000 generasi mengikuti proses yang sama dengan pencarian 100 generasi pada 100 generasi pertamanya, kemudian melanjutkan pencarian. Elitisme menjaga nilai terbaik agar tidak turun. Kenaikan rata-rata dari 800 ke 1.000 generasi sebesar 2,33 poin seluruhnya berasal dari *seed* 66, yang naik dari 440 menjadi 454. Nilai maksimum 467 sudah ditemukan pada 600 generasi, sementara beberapa *run* masih membaik setelahnya.

2.8.2 Contoh Perkembangan pada Generasi yang Berbeda

Gambar 2.2 dan Gambar 2.3 memperlihatkan pencarian dengan *seed* 30 dan populasi yang sama, yaitu 20 individu. Gambar 2.2 berhenti setelah 100 generasi, sedangkan Gambar 2.3 meneruskan pencarian sampai 1.000 generasi. Garis biru menunjukkan nilai individu terbaik dan garis merah menunjukkan rata-rata nilai seluruh populasi.



Gambar 2.2: Kurva GA dengan 20 individu dan 100 generasi pada *seed* 30.



Gambar 2.3: Kurva GA dengan 20 individu dan 1.000 generasi pada *seed* 30.

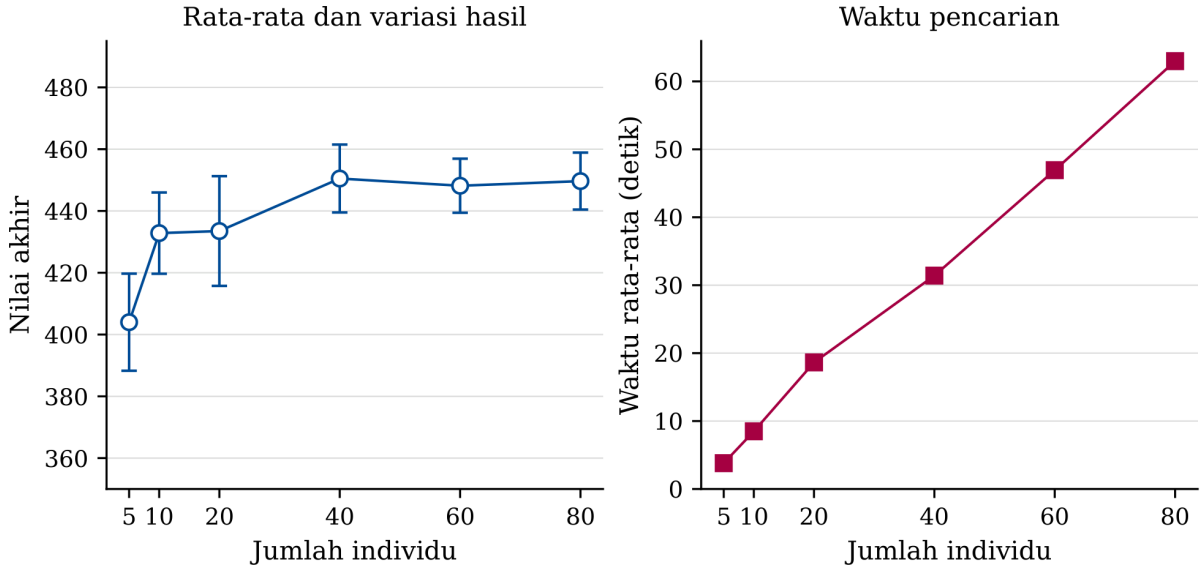
Keduanya berawal dari nilai terbaik 418. Pada 100 generasi, nilai tersebut belum meningkat. Pencarian yang lebih panjang kemudian mencapai 467. Garis rata-rata tetap bergerak naik dan turun karena isi populasi berubah, sedangkan individu terbaik dipertahankan melalui elitisme.

2.8.3 Pengaruh Jumlah Individu dalam Populasi

Jumlah generasi ditahan pada 400. Populasi yang lebih besar memberi lebih banyak kandidat sekaligus menambah jumlah evaluasi. Sebagai contoh, populasi 20 dengan 400 generasi memerlukan 8.020 evaluasi individu, termasuk evaluasi populasi awal.

Populasi	Rata-rata	Simp. baku	Min–maks	Durasi rata-rata (detik)
5	404,00	15,72	384–421	3,80
10	432,83	13,15	418–455	8,49
20	433,50	17,76	414–455	18,64
40	450,50	10,99	435–463	31,41
60	448,17	8,75	435–456	46,93
80	449,67	9,22	432–456	62,98

Tabel 2.5: Hasil GA pada 400 generasi. Setiap baris merangkum enam *run*.



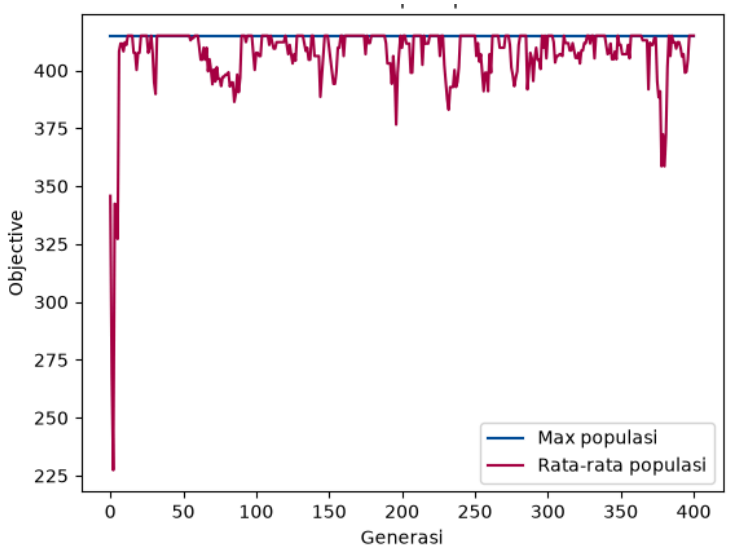
Gambar 2.4: Pengaruh ukuran populasi terhadap nilai akhir dan durasi. Batang galat menunjukkan simpangan baku sampel.

Populasi 40 menghasilkan rata-rata tertinggi pada pengujian ini, yaitu 450,50 dengan durasi rata-rata 31,41 detik. Populasi 60 dan 80 memerlukan durasi lebih lama, tetapi rata-ratanya berada pada 448,17 dan 449,67. Populasi yang lebih besar ternyata belum tentu memberi nilai akhir lebih tinggi.

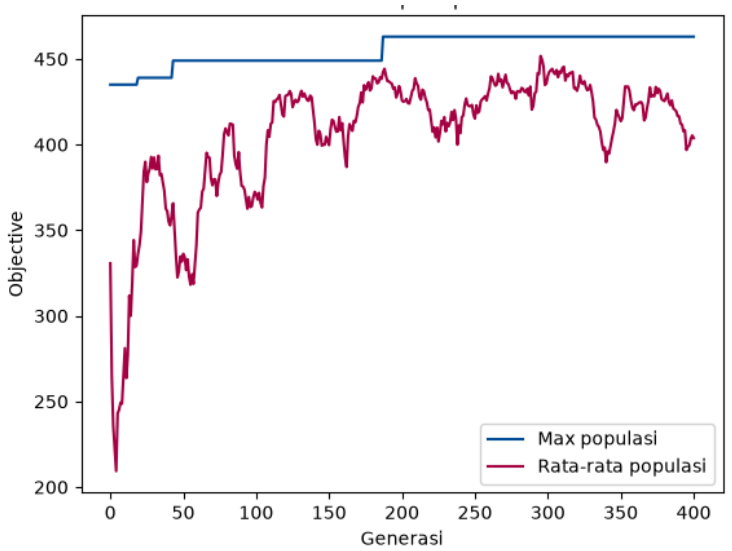
Populasi 60 memiliki simpangan baku paling kecil, yaitu 8,75, sehingga hasil keenam *run* lebih berdekatan. Namun, nilai rata-ratanya lebih rendah daripada populasi 40. Pemilihan parameter perlu mempertimbangkan kualitas, durasi, dan konsistensi. Perubahan populasi juga dapat mengubah individu terbaik sejak inisialisasi, sehingga selisih hasil tidak seluruhnya berasal dari perbaikan selama evolusi.

2.8.4 Pengaruh Populasi Sejak *State* Awal

Ukuran populasi memengaruhi jumlah kandidat sejak pencarian dimulai. Gambar 2.5 dan Gambar 2.6 membandingkan populasi 5 dan 40 individu dengan batas 400 generasi pada *seed* 45. Nilai awal yang ditampilkan adalah nilai terbaik di antara seluruh individu pada populasi awal.



Gambar 2.5: Kurva GA dengan 5 individu dan 400 generasi pada *seed* 45.



Gambar 2.6: Kurva GA dengan 40 individu dan 400 generasi pada *seed* 45.

Pada populasi 5, nilai 415 tidak berubah. Populasi 40 berawal dari 435 dan meningkat menjadi 463. Selisih akhirnya 48, yang terdiri atas 20 poin perbedaan awal dan 28 poin peningkatan selama pencarian. Pada contoh *seed* 45 ini, kualitas populasi awal dan proses perbaikannya sama-sama berpengaruh pada hasil akhir.

2.9 Hasil Pencarian Hill-Climbing

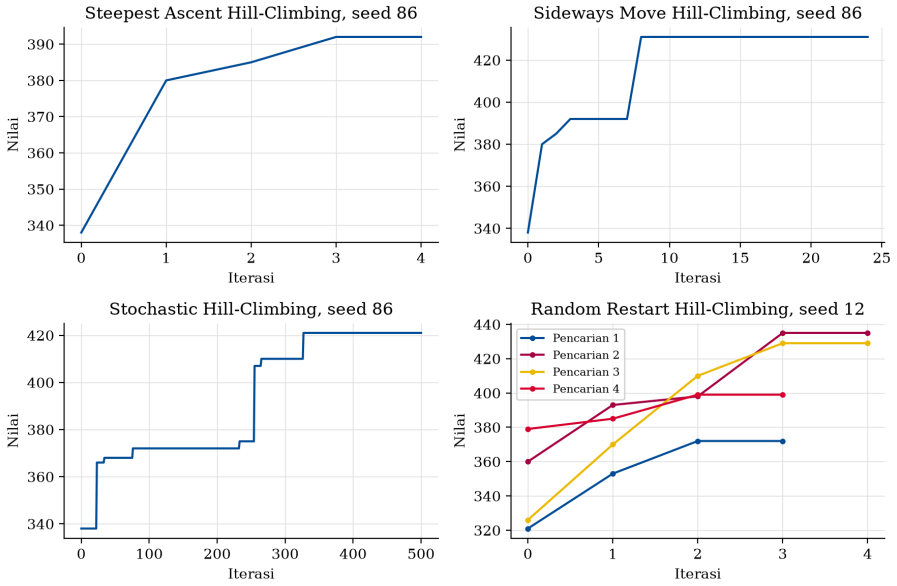
2.9.1 Ringkasan Empat Varian

Seed	Awal	Steepest Ascent	Sideways Move	Stochastic	Random Restart
12	321	372	397	363	435
30	298	384	384	358	393
45	415	415	415	415	428
66	342	372	372	350	425
73	334	353	388	353	390
86	338	392	431	421	406

Tabel 2.6: Nilai awal dan akhir empat varian Hill-Climbing. Seluruh varian memakai *state* awal yang sama per *seed*.

Seed	Steepest Ascent	Sideways Move	Stochastic	Iterasi tiap pencarian Random Restart
12	3	24	500	3, 4, 4, 3
30	5	24	500	5, 1, 5, 2
45	1	20	500	1, 4, 5, 2
66	3	22	500	5, 8, 3, 4
73	2	23	500	2, 3, 5, 3
86	4	24	500	4, 3, 7, 1

Tabel 2.7: Jumlah iterasi empat varian Hill-Climbing. Random Restart Hill-Climbing menjalankan satu pencarian awal dan tiga *restart*.

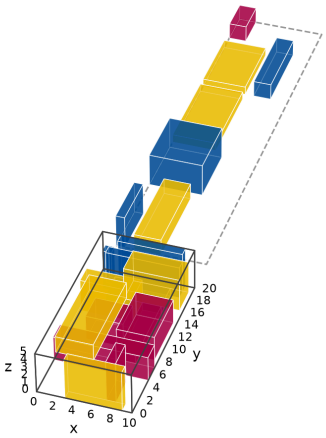


Gambar 2.7: Contoh kurva Hill-Climbing. Setiap pencarian Random Restart Hill-Climbing digambar terpisah. Rentang sumbu iterasi mengikuti panjang pencarian masing-masing varian.

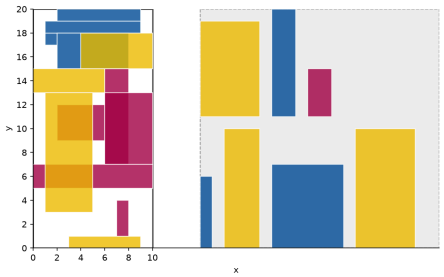
2.9.2 Contoh *State* pada Steepest Ascent Hill-Climbing

Pada *seed* 86, nilai *State* meningkat melalui urutan 338, 380, 385, dan 392. Pencarian berhenti ketika pemeriksaan berikutnya tidak menemukan *neighbor* yang lebih baik. Muatan akhirnya terdiri atas 14 paket dengan berat 61.

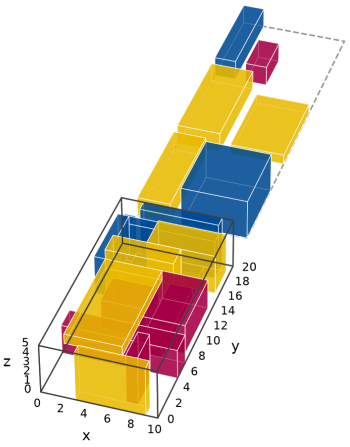
State awal



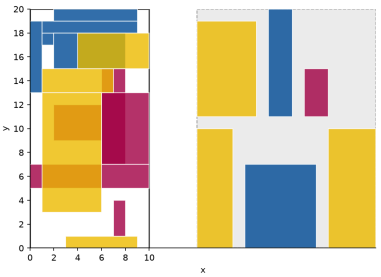
Tampilan atas *state* awal



State akhir



Tampilan atas *state* akhir



Gambar 2.8: *State* awal dan akhir Steepest Ascent Hill-Climbing pada *seed* 86. Warna mengikuti tampilan program.

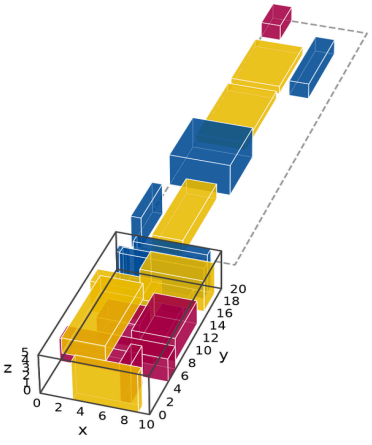
Paket yang baru masuk adalah P01 dan P11. Paket yang dikeluarkan adalah P16.

Steepest Ascent Hill-Climbing hanya menerima peningkatan nilai. Karena itu, pencarian pada contoh ini berhenti di nilai 392, meskipun algoritma lain dapat menempuh jalur berbeda dari *State* awal yang sama.

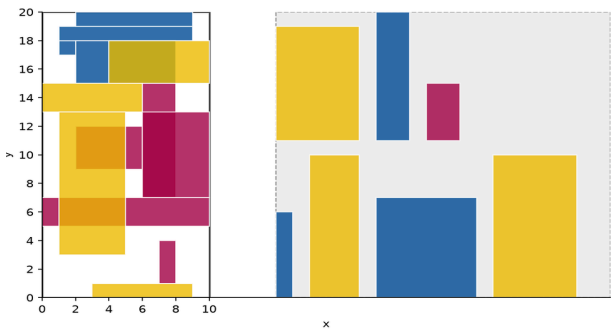
2.9.3 Contoh *State* pada Sideways Move Hill-Climbing

Sideways Move Hill-Climbing mencapai nilai 431 dari nilai awal 338 dalam 24 iterasi pada *seed* 86. Pencarian berhenti setelah menerima 20 sideways *move*, yaitu batas yang ditetapkan. Muatan akhirnya terdiri atas 14 paket dengan berat 65, tepat pada kapasitas truk.

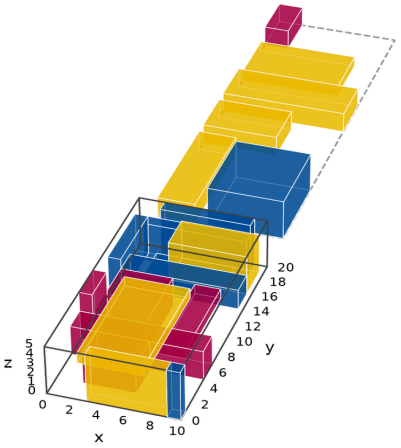
State awal



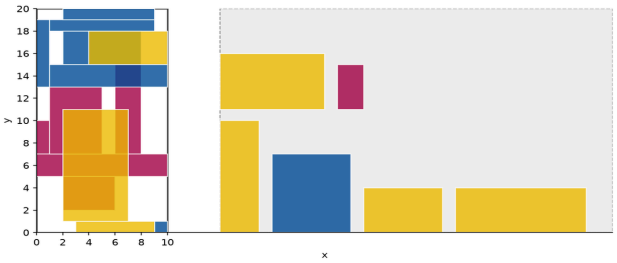
Tampilan atas *state* awal



State akhir



Tampilan atas *state* akhir



Gambar 2.9: *State* awal dan akhir Sideways Move Hill-Climbing pada *seed* 86. Warna mengikuti tampilan program.

Paket yang baru masuk adalah P01, P11, dan P19. Paket yang dikeluarkan adalah P14 dan P16.

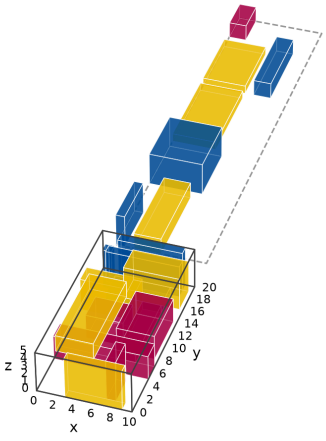
Sideways *move* memungkinkan perubahan posisi tanpa langsung menaikkan nilai. Perubahan posisi ini dapat membuka kesempatan untuk perbaikan pada langkah berikutnya, tetapi membutuhkan evaluasi *neighbor* tambahan.

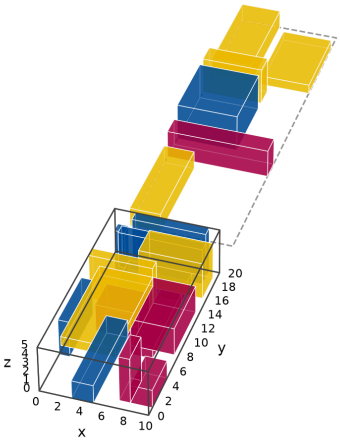
2.9.4 Contoh *State* pada Stochastic Hill-Climbing

Pada *seed* 86, Stochastic Hill-Climbing meningkatkan nilai dari 338 menjadi 421 selama 500 iterasi. Algoritma hanya mengambil satu *neighbor* secara acak pada tiap iterasi, sehingga jalur pencariannya berbeda dari Steepest Ascent Hill-Climbing. Muatan akhirnya terdiri atas 14 paket dengan berat 63.

State awal

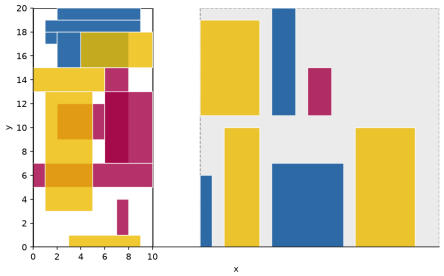
State akhir

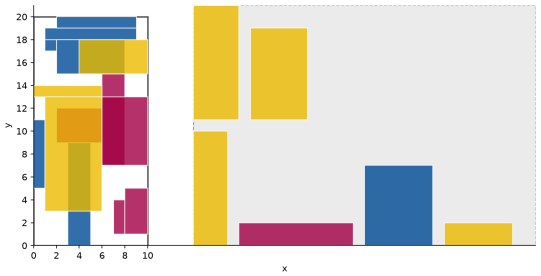




Tampilan atas *state* awal

Tampilan atas *state* akhir





Gambar 2.10: *State* awal dan akhir Stochastic Hill-Climbing pada *seed* 86. Warna mengikuti tampilan program.

Paket yang baru masuk adalah P01, P11, P19, dan P20. Paket yang dikeluarkan adalah P08, P14, dan P16.

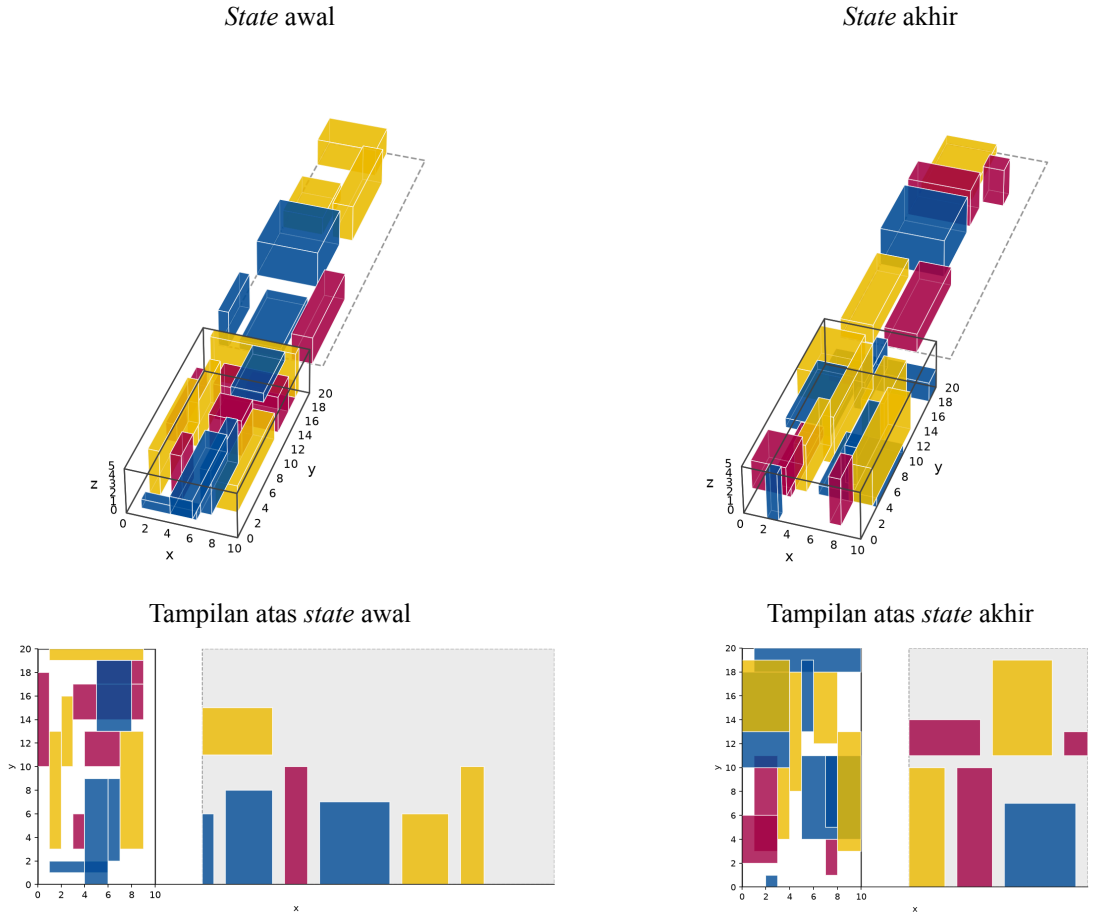
Nilai 421 pada *run* ini lebih tinggi daripada hasil Steepest Ascent Hill-Climbing. Namun, rata-rata keenam *run* Stochastic Hill-Climbing justru lebih rendah, dengan hasil yang lebih bervariasi. Pengambilan satu kandidat per iterasi membuat durasi pencariannya relatif singkat.

2.9.5 Contoh *State* dan *Restart* pada Random Restart Hill-Climbing

Random Restart Hill-Climbing menjalankan empat pencarian dari *state* awal yang berbeda. Satu pencarian awal diikuti tiga *restart*. Pada *seed* 12, Tabel 2.8 menunjukkan bahwa nilai awal yang lebih rendah masih dapat menghasilkan nilai akhir yang lebih tinggi. Hasil keseluruhan diambil dari nilai terbaik di antara keempat pencarian.

Pencarian	Nilai awal	Nilai akhir	Iterasi
1	321	372	3
2	360	435	4
3	326	429	4
4	379	399	3

Tabel 2.8: Rincian Random Restart Hill-Climbing pada *seed* 12. Nilai tiap pencarian diambil dari hasil *run* per *seed* yang disimpan di folder `output/` (`result.json`).



Gambar 2.11: *State* awal pencarian pertama dan hasil terbaik Random Restart Hill-Climbing pada *seed* 12.

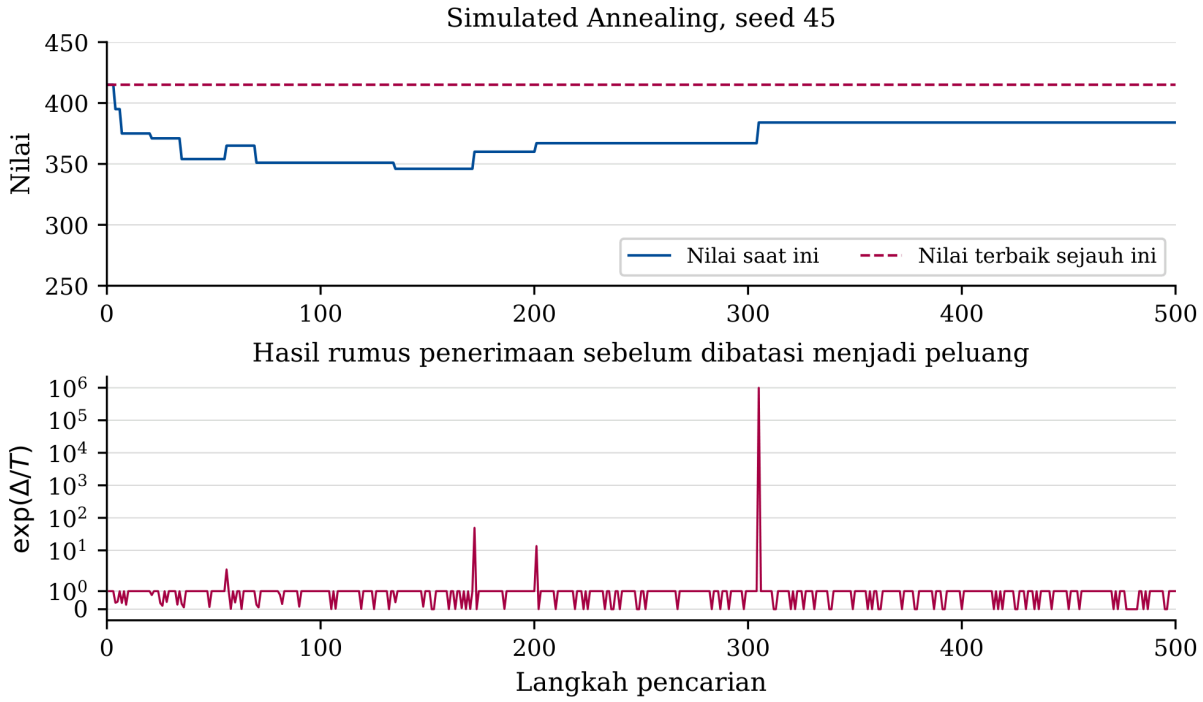
Pencarian mengembalikan nilai 435 dari nilai awal keseluruhan 321. Paket yang baru masuk adalah P01, P05, P14, P16, dan P17. Paket yang dikeluarkan adalah P02, P10, P18, dan P20. *Restart* memberi jalur lain ketika pencarian sebelumnya berhenti, dengan biaya durasi untuk menjalankan beberapa pencarian.

2.10 Hasil Pencarian Simulated Annealing

2.10.1 Ringkasan Nilai, Waktu, dan Kondisi Pencarian

Seed	Awal	Akhir	Terbaik	Periode *stuck*	Frekuensi *stuck* (%)	Durasi (detik)
12	321	379	379	10	45,80	87,69
30	298	410	410	7	8,20	87,11
45	415	384	415	9	19,80	104,74
66	342	399	399	11	42,80	115,34
73	334	385	385	6	4,80	39,97
86	338	425	425	1	34,40	107,03

Tabel 2.9: Hasil SA pada 500 iterasi. Frekuensi *stuck* dihitung dari jumlah iterasi dalam kondisi *stuck* dibagi 500. Periode *stuck* diambil dari hasil *run* per *seed* yang disimpan di folder `output/` (`result.json`).



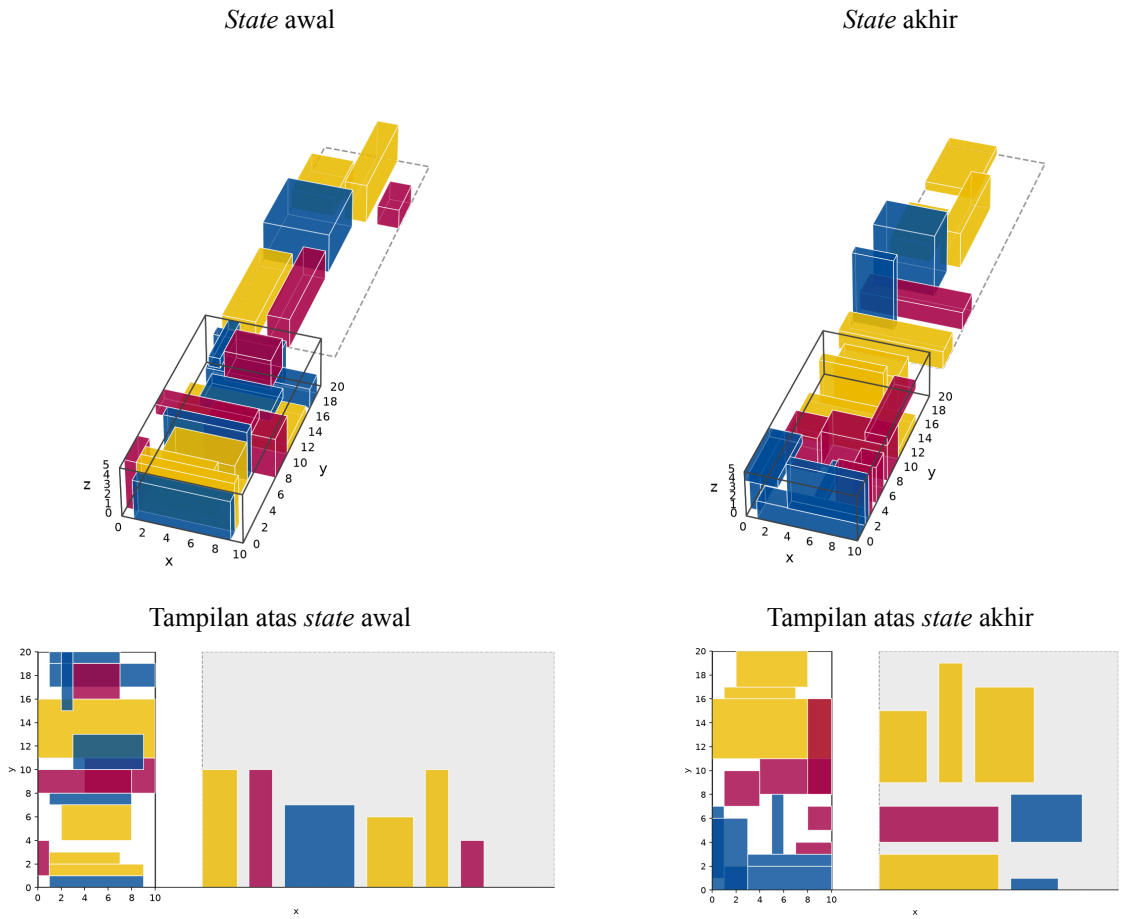
Gambar 2.12: Perkembangan SA pada *seed* 45. Grafik bawah menampilkan $exp(\Delta/T)$ pada skala yang memadatkan rentang nilai yang besar dengan batas tampilan 10^6. Hasil rumus sebelum dibatasi menjadi peluang dapat melebihi satu dan dibedakan dari peluang penerimaan.

SA menghasilkan rata-rata nilai akhir 397,00 pada enam *run*. Nilai akhir tertingginya adalah 425 pada *seed* 86. Hasil pada *seed* 45 menunjukkan bahwa penerimaan kandidat yang lebih buruk dapat membuat nilai akhir lebih rendah daripada nilai awal. Rinciannya dibahas pada bagian berikut.

Kondisi *stuck* tercatat pada 4,80% sampai 45,80% iterasi. Pemeriksaan kondisi ini dilakukan selama pencarian dan ikut menambah durasi SA.

2.10.2 Ketika Nilai Akhir Lebih Rendah daripada Nilai Terbaik

Pada *seed* 45, *State* awal bernilai 415 dan tetap menjadi *State* terbaik hingga pencarian selesai. Selama *run*, SA menerima enam *move* yang menurunkan nilai dan akhirnya berhenti pada nilai 384.



Gambar 2.13: *State* awal yang sekaligus menjadi *state* terbaik dan *state* akhir SA pada *seed* 45.

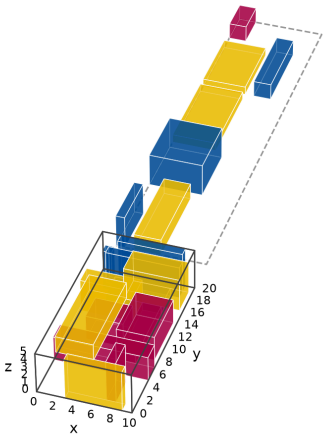
Paket yang baru masuk adalah P20. Paket yang dikeluarkan adalah P05 dan P18.

Perbandingan pada Tabel 2.11 menggunakan nilai *State* akhir, sehingga *run* ini dicatat dengan nilai 384. Jika yang dinilai adalah *State* terbaik yang pernah ditemukan, metrik `best_score` memberikan hasil berbeda. Pada enam *seed*, rata-rata `best_score` SA adalah 402,17, sedangkan rata-rata nilai akhirnya 397,00.

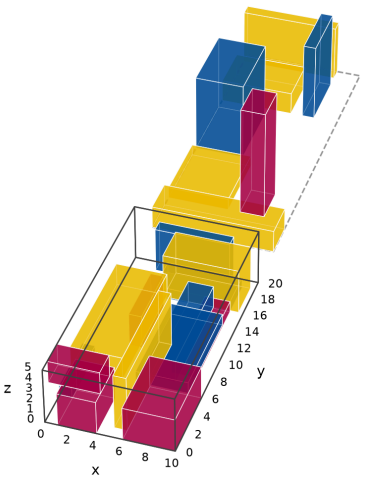
2.10.3 Contoh Pencarian SA yang Meningkatkan Nilai

Pada *seed* 86, SA meningkatkan nilai dari 338 menjadi 425 dengan jumlah paket tetap 13. Perubahan pilihan paket menaikkan berat muatan dari 60 menjadi 65. *State* akhir pada *run* ini sekaligus merupakan *State* terbaik yang ditemukan.

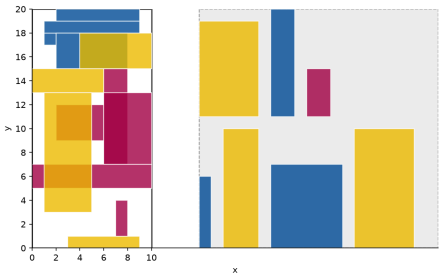
State awal



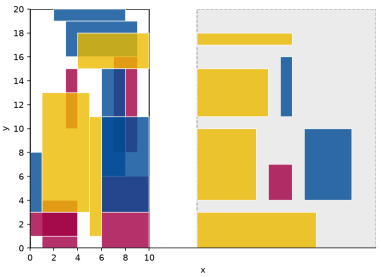
State akhir



Tampilan atas *state* awal



Tampilan atas *state* akhir



Gambar 2.14: *State* awal dan akhir SA pada *seed* 86, dengan nilai akhir 425.

Paket yang baru masuk adalah P01, P11, P19, dan P20. Paket yang dikeluarkan adalah P07, P08, P14, dan P15.

Sebanyak 34,40% iterasi pada *run* ini tercatat dalam kondisi *stuck*. Meskipun demikian, nilai akhirnya menjadi yang tertinggi di antara enam *run* SA.

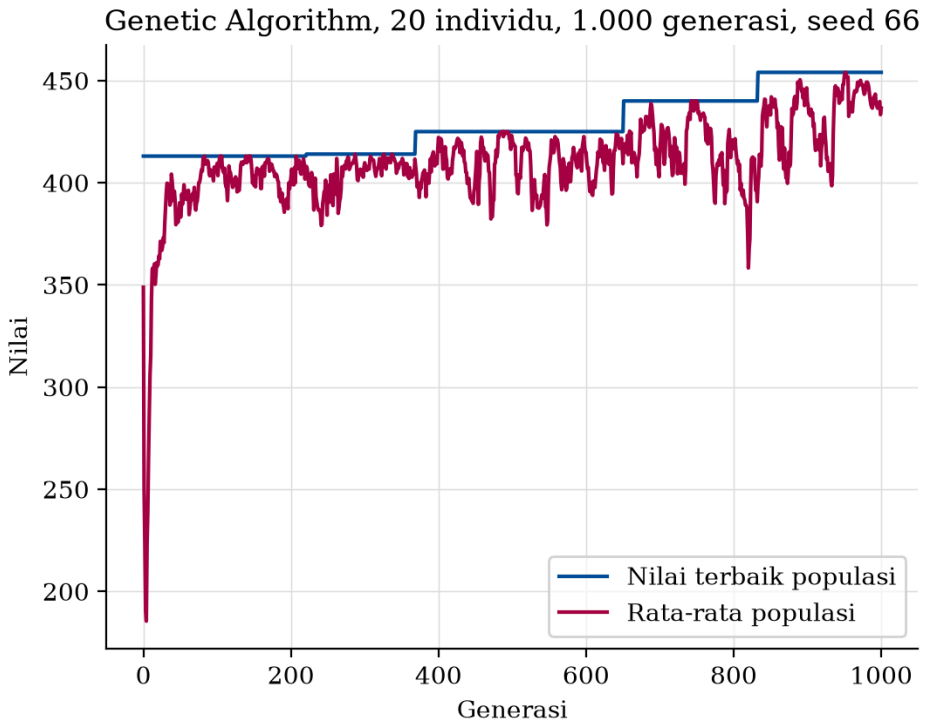
2.11 Hasil Pencarian Genetic Algorithm

2.11.1 Perubahan Nilai pada Enam *Run*

GA dengan 20 individu dan 1.000 generasi menghasilkan rata-rata nilai akhir tertinggi, yaitu 452,17. Selisihnya dengan konfigurasi 40 individu dan 400 generasi hanya 1,67 poin. Peningkatan pada tiap *seed* berbeda-beda. Nilai awal yang dicatat merupakan nilai individu terbaik di antara 20 kandidat pada populasi awal.

Seed	Nilai awal	Nilai akhir	Kenaikan	Durasi (detik)
12	434	435	1	36,12
30	418	467	49	41,16
45	428	454	26	37,34
66	413	454	41	32,78
73	412	448	36	40,65
86	449	455	6	48,38

Tabel 2.10: Hasil GA dengan 20 individu dan 1.000 generasi.



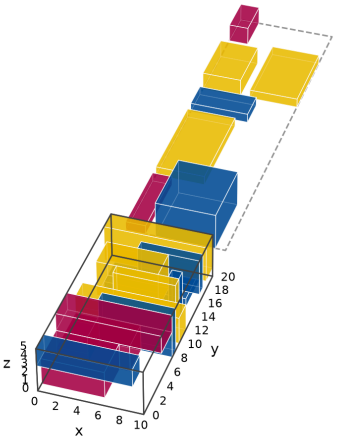
Gambar 2.15: Kurva nilai terbaik dan rata-rata populasi pada *seed* 66.

Pada seed 30 (Gambar 2.3), nilai terbaik meningkat dari 418 menjadi 467 dan pertama kali mencapai 467 pada generasi 521. Pada seed 66 (Gambar 2.15), nilai terbaik naik bertahap dari 413 menjadi 454 dan pertama kali mencapai 454 pada generasi 833. Pada seed 12, kenaikannya hanya satu poin. Individu awal yang sudah baik tetap membutuhkan neighbor atau hasil crossover yang sesuai agar dapat diperbaiki. Garis rata-rata bergerak mengikuti pergantian individu, sedangkan garis terbaik tidak turun karena elitisme.

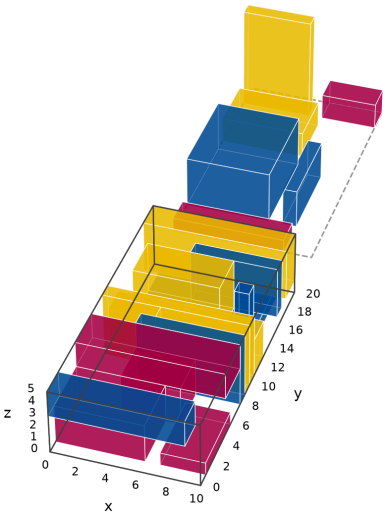
2.11.2 Contoh *State* Terbaik yang Ditemukan GA

Pada *seed* 30, GA dengan 20 individu dan 1.000 generasi meningkatkan nilai dari 418 menjadi 467. Muatan bertambah satu paket, dari 13 menjadi 14, dengan kenaikan berat hanya dua unit, dari 60 menjadi 62. Perubahan pilihan dan susunan paket menghasilkan tambahan nilai sebesar 49 poin.

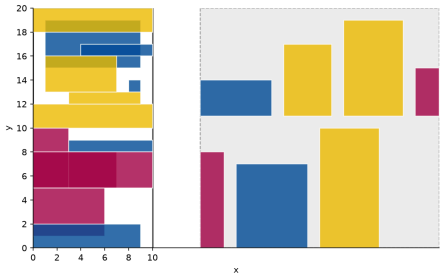
State awal



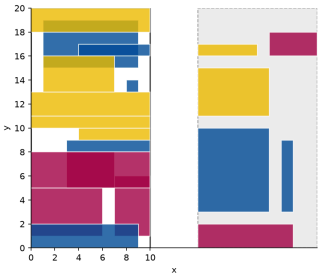
State akhir



Tampilan atas *state* awal



Tampilan atas *state* akhir



Gambar 2.16: *State* awal dan akhir GA dengan 20 individu dan 1.000 generasi pada *seed* 30.

Paket yang baru masuk adalah P11.

Pada hasil ini, beberapa paket belum berhasil dimasukkan. Berat muatan baru mencapai 62 dari kapasitas 65, tetapi ruang yang tampak kosong belum tentu cukup untuk paket yang tersisa. Ukuran, orientasi, kebutuhan penopang, dan status pecah-belah tetap membatasi penempatan.

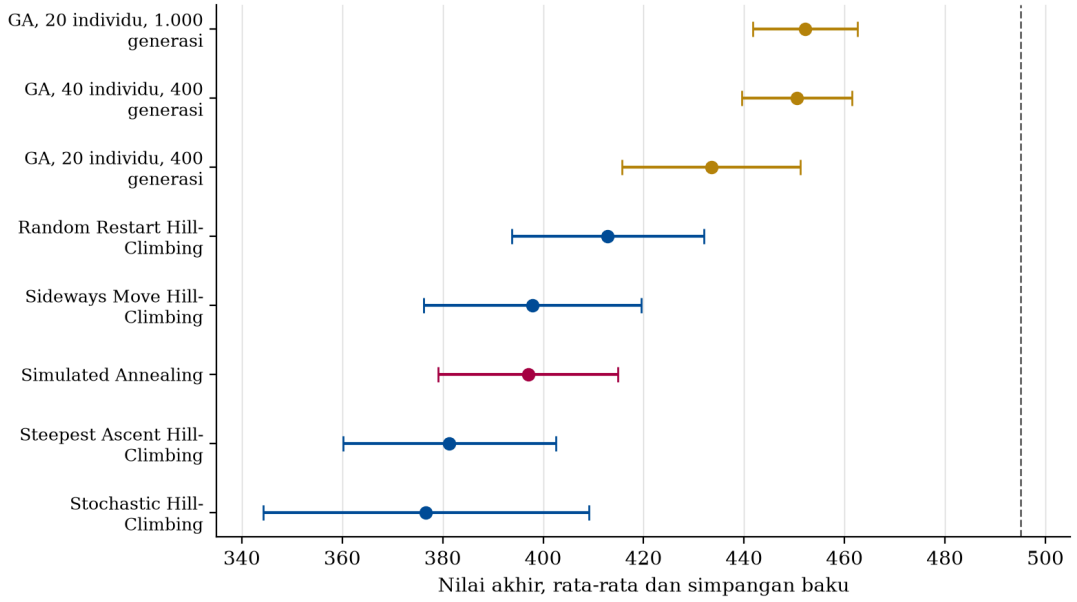
2.12 Perbandingan dan Analisis Akhir

2.12.1 Kualitas Hasil, Waktu, dan Konsistensi

Tabel 2.11 membandingkan HC dan SA dengan tiga konfigurasi GA: 20 individu dan 400 generasi sebagai acuan, 40 individu dan 400 generasi, serta 20 individu dan 1.000 generasi. Seluruh nilai yang dibandingkan adalah nilai *State* akhir.

Algoritma	**Rata-rata ± simp. baku**	**Min–maks**	**Durasi rata-rata (detik)**
Steepest Ascent Hill-Climbing	$381,33 \pm 21,13$	353–415	6,22
Sideways Move Hill-Climbing	$397,83 \pm 21,68$	372–431	63,67
Stochastic Hill-Climbing	$376,67 \pm 32,38$	350–421	1,25
Random Restart Hill-Climbing	$412,83 \pm 19,14$	390–435	29,90
Simulated Annealing	$397,00 \pm 17,85$	379–425	90,31
GA, 20 individu dan 400 generasi	$433,50 \pm 17,76$	414–455	18,64
GA, 40 individu dan 400 generasi	$450,50 \pm 10,99$	435–463	31,41
GA, 20 individu dan 1.000 generasi	$452,17 \pm 10,46$	435–467	39,41

Tabel 2.11: Perbandingan nilai akhir pada enam *seed*.



Gambar 2.17: Rata-rata nilai akhir dan simpangan baku. Garis putus-putus menunjukkan batas atas perhitungan berdasarkan berat.

GA dengan 20 individu dan 1.000 generasi menghasilkan rata-rata 452,17. GA dengan 40 individu dan 400 generasi menghasilkan rata-rata yang dekat, yaitu 450,50, dengan durasi lebih singkat. Stochastic Hill-Climbing paling cepat, tetapi hasilnya lebih menyebar. Random Restart Hill-Climbing memperoleh rata-rata tertinggi di antara varian HC. Pilihan algoritma pada kasus ini bergantung pada nilai solusi yang diinginkan dan waktu pencarian yang tersedia.

Durasi tidak hanya ditentukan oleh jumlah iterasi. Steepest Ascent Hill-Climbing mengevaluasi 144–892 *neighbor* per *run*, Sideways Move Hill-Climbing 2.969–6.343, `Random` Restart Hill-Climbing 2.339–4.478, dan Stochastic Hill-Climbing tepat 500. GA mengevaluasi 8.020 individu pada 20 individu dan 400 generasi, 16.040 pada 40 individu dan 400 generasi, serta 20.020 pada 20 individu dan 1.000 generasi. Jumlah ini menghitung evaluasi individu atau kandidat, bukan seluruh percobaan *move* dan pemeriksaan validasi.

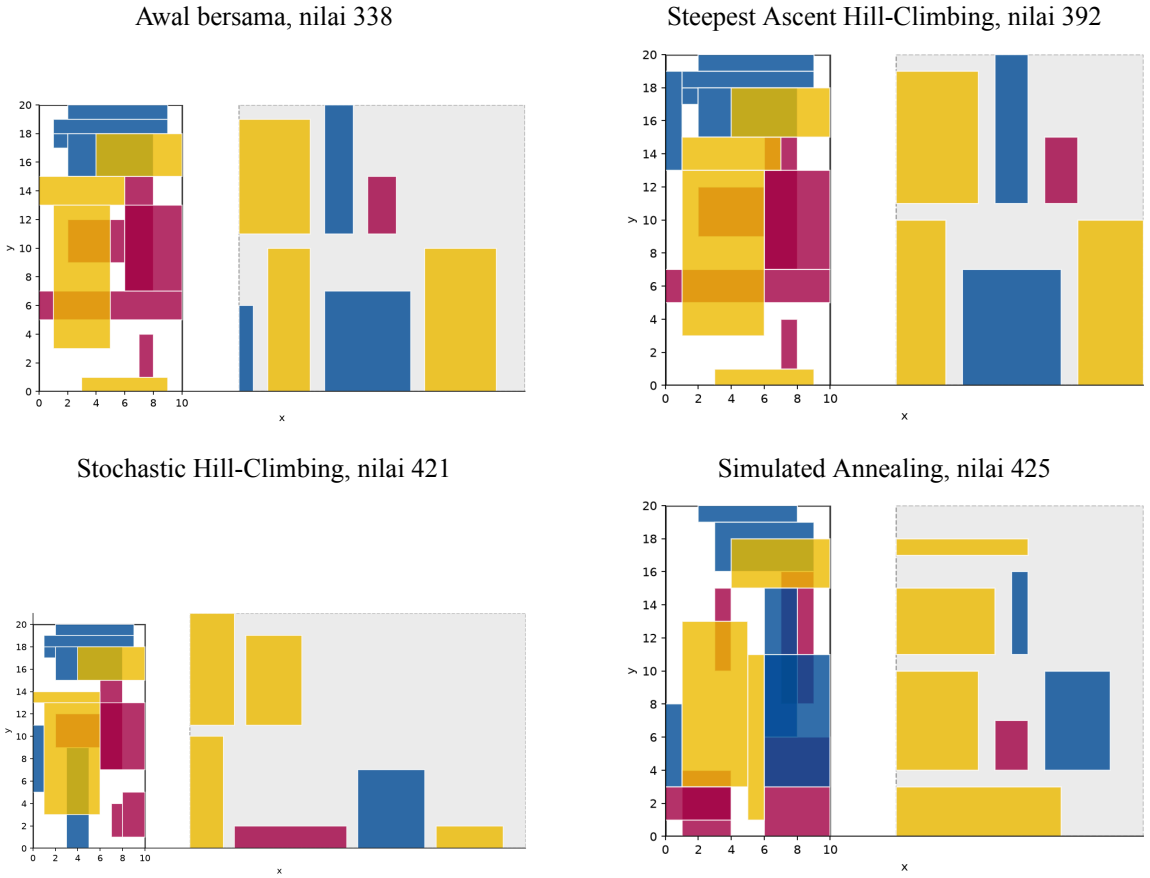
Biaya tiap evaluasi juga berbeda. GA dengan 8.020 evaluasi selesai dalam rata-rata 18,64 detik, sedangkan Sideways Move Hill-Climbing memerlukan 63,67 detik. SA memerlukan rata-rata 90,31 detik. Selain menilai 500 kandidat utama, SA memeriksa kondisi *stuck* pada *state* awal dan setelah setiap kandidat diterima. Fungsi `improving_neighbor_exists` mencoba penambahan paket dan *swap* yang dapat menaikkan nilai, lalu berhenti segera setelah menemukan perbaikan. Jika tidak ada perbaikan, seluruh kemungkinan yang relevan harus diperiksa. Pemeriksaan tambahan ini ikut masuk durasi SA. Jumlahnya tidak tercakup dalam angka 500 kandidat utama.

Dari sisi konsistensi, Stochastic Hill-Climbing paling menyebar dengan simpangan baku 32,38 dan rentang 350–421. Steepest Ascent Hill-Climbing, Sideways Move Hill-Climbing, dan Random Restart Hill-Climbing memiliki simpangan baku 21,13, 21,68, dan 19,14, sedangkan SA 17,85. Hasil HC sangat bergantung pada State awal. Pada seed 45, Steepest Ascent Hill-Climbing, Sideways Move Hill-Climbing, dan Stochastic Hill-Climbing berakhir pada nilai awal 415. GA dengan 40 individu dan 400 generasi serta GA dengan 20 individu dan 1.000 generasi paling konsisten, dengan simpangan baku 10,99 dan 10,46 serta nilai minimum 435. GA dengan 20 individu dan 400 generasi lebih menyebar, yaitu 17,76 dengan rentang 414–455.

Rentang rata-rata ± simpangan baku beberapa algoritma saling tumpang tindih. Tumpang tindih ini bukan pengujian signifikansi. Dengan satu kasus, enam *seed* per konfigurasi, dan tanpa uji perbedaan, peringkat pada Tabel 2.11 merupakan perbandingan deskriptif hasil pengujian, bukan bukti keunggulan umum suatu algoritma.

2.12.2 Perbandingan *State* Akhir dari *State* Awal yang Sama

HC dan SA pada *seed* 86 menggunakan *state* awal yang sama dengan nilai 338. Gambar 2.18 menampilkan empat tampilan atas, yaitu *state* awal, hasil Steepest Ascent Hill-Climbing, hasil Stochastic Hill-Climbing, dan hasil SA. Bagian di sebelah kanan setiap gambar memuat paket yang berada di luar truk.



Gambar 2.18: Perbandingan tampilan atas dari *state* awal yang sama pada *seed* 86.

Posisi, orientasi, dan pilihan paket pada *state* akhir dapat berbeda walaupun pencarian dimulai dari *state* yang sama. Tampilan ruang membantu melihat bentuk perubahan, sedangkan nilai akhir menunjukkan manfaat paket yang dibawa. Luas bidang yang terlihat terisi tidak langsung menyatakan nilai *state* atau sisa kapasitas berat.

Dari *State* awal yang sama, Stochastic Hill-Climbing dan SA mencapai nilai yang lebih tinggi daripada Steepest Ascent Hill-Climbing pada *seed* ini. Perbedaan posisi dan orientasi terlihat pada tampilan atas, sedangkan kelayakan susunannya diperiksa menggunakan data tiga dimensi.

2.12.3 Kedekatan dengan Nilai Terbaik dan Batas Kesimpulan

Batas atas nilai muatan dihitung dengan memilih paket berdasarkan berat dan nilainya saja. Setiap paket dapat dipilih satu kali dengan total berat paling banyak 65. Kendala geometris dan penopang diabaikan pada perhitungan ini.

$$U = max \sum_i v_i x_i, \qquad \sum_i w_i x_i \leq 65, \qquad x_i \in \{0, 1\}.$$

Perhitungan ini menghasilkan batas atas 495, sedangkan GA telah menemukan State valid bernilai 467. Nilai optimum global berada di antara keduanya, yaitu $467 \leq f^* \leq 495$.

Algoritma	Hasil tertinggi	Selisih ke batas atas (%)
Steepest Ascent Hill-Climbing	415	16,16
Sideways Move Hill-Climbing	431	12,93
Stochastic Hill-Climbing	421	14,95
Random Restart Hill-Climbing	435	12,12
Simulated Annealing	425	14,14
GA, 20 individu dan 1.000 generasi	467	5,66

Tabel 2.12: Hasil tertinggi yang ditemukan pada enam *seed* dibandingkan dengan batas atas 495.

Hasil terbaik GA berjarak 28 poin atau 5,66% dari batas atas 495. Karena acuan ini mengabaikan kendala geometris, selisih tersebut belum menunjukkan jarak terhadap optimum global.

Algoritma	Ke batas atas 495 (%)	Ke hasil terbaik 467 (%)
Steepest Ascent Hill-Climbing	22,96	18,34
Sideways Move Hill-Climbing	19,63	14,81
Stochastic Hill-Climbing	23,91	19,34
Random Restart Hill-Climbing	16,60	11,60
Simulated Annealing	19,80	14,99
GA (20 individu, 1.000 generasi)	8,65	3,18

Tabel 2.13: Selisih rata-rata nilai akhir terhadap batas atas 495 dan hasil terbaik 467.

Pada kedua acuan, GA memiliki selisih rata-rata terkecil. Perhitungan ini merangkum keenam *run*, sehingga berbeda dari perbandingan yang memakai satu hasil tertinggi.

Seluruh hasil berasal dari satu kasus dan enam *seed*. Parameter GA juga dipilih dan dibandingkan pada data yang sama. Selain itu, GA memulai pencarian dari sebuah populasi, sedangkan HC dan SA memulai dari satu *state*. Batas iterasi serta biaya tiap iterasi tidak dibuat sama. Kesimpulan yang lebih luas memerlukan kasus tambahan dan pembandingan dengan anggaran durasi atau jumlah evaluasi yang sebanding.

2.13 Bonus: Fungsi Objektif Berbasis ETA

Fungsi objektif dasar hanya memperhitungkan nilai paket. Dalam pengiriman, paket dengan ETA kecil perlu lebih cepat sampai sehingga lebih diutamakan untuk dimuat. Pada fungsi objektif kedua, nilai setiap paket di dalam truk dibagi dengan 1 + ETA_i/2. Pembagian ETA dengan 2 menjaga agar pengaruh ETA tidak terlalu besar sehingga nilai paket tetap menjadi pertimbangan utama. Fungsi ini tidak menambah variabel baru, sedangkan kendala dan langkah pencarian tetap sama.

ETA	Bobot 1/(1 + ETA/2)
0	1,00
1	0,67
2	0,50
3	0,40

Tabel 2.14: Bobot paket berdasarkan ETA pada fungsi objektif kedua.

Pengujian fungsi objektif kedua dijalankan pada kasus benchmark20 dengan seed 12 dengan memilih objektif eta pada konfigurasi run. Pada ketiga algoritma, skor f_eta(s) meningkat dari state awal ke state akhir dan state akhir tetap valid. Pengujian ini hanya mencakup satu seed pada satu kasus sehingga hanya menunjukkan bahwa fungsi objektif kedua dapat dijalankan, bukan perbandingan kualitas antar kedua algoritma.

Tabel 2.15: Hasil fungsi objektif kedua pada kasus 20 paket dengan seed 12.

Algoritma	Skor eta awal	Skor eta akhir	Durasi (detik)	Nilai f(s) dari *state* akhir
Steepest Ascent Hill-Climbing	233,07	267,90	7,4	372
Simulated Annealing (500 iterasi)	233,07	244,20	21,6	346
Genetic Algorithm (20 individu, 400 generasi)	305,97	341,50	20,8	442

BAB 3
Kesimpulan dan Saran

3.1 Kesimpulan

Program menerapkan pencarian susunan paket menggunakan HC, SA, dan GA dengan representasi *state* serta fungsi objektif yang sama. Validasi memastikan paket berada dalam batas truk, tidak beririsan, memiliki penopang yang sah, dan memenuhi kapasitas berat sebelum nilainya dihitung.

Pada kasus 20 paket dengan enam *seed*, GA dengan 20 individu dan 1.000 generasi menghasilkan rata-rata nilai akhir tertinggi, yaitu 452,17, dan nilai maksimum 467. GA dengan 40 individu dan 400 generasi menghasilkan rata-rata 450,50, hanya terpaut 1,67 poin, dengan durasi lebih singkat. Nilai terbaik 467 berjarak 5,66% dari batas atas berdasarkan berat, yaitu 495. Optimum global kasus ini belum diketahui.

Stochastic Hill-Climbing paling cepat, dengan durasi rata-rata 1,25 detik, tetapi memiliki simpangan baku terbesar, yaitu 32,38. Random Restart Hill-Climbing menghasilkan rata-rata tertinggi di antara varian HC, yaitu 412,83. Pada SA, penerimaan penurunan nilai membuat *State* akhir dapat berbeda dari *State* terbaik selama pencarian.

Penambahan generasi GA meningkatkan rata-rata pada rentang pengujian, sedangkan penambahan populasi tidak selalu memberi hasil lebih tinggi. Perbandingan ini berlaku pada satu kasus serta konfigurasi yang diuji.

Untuk bonus dari tugas besar ini, kami mengerjakan 2 dari 3 bonus. Bonus 1 (fungsi objektif berbasis ETA) dan bonus 2 (keempat varian Hill-Climbing) telah dipenuhi.

3.2 Saran

Eksperimen berikutnya perlu mencakup ukuran truk, jumlah paket, dan komposisi barang yang berbeda. Parameter sebaiknya dipilih pada sebagian kasus atau *seed*, lalu diuji pada kasus atau *seed* lain agar kesimpulan tidak hanya mengikuti data pemilihan parameter.

Perbandingan efisiensi dapat diperkuat dengan anggaran evaluasi atau durasi yang sebanding. Pemeriksaan *neighbor* dan validasi geometris dapat dioptimalkan untuk kasus yang lebih besar. Untuk SA, biaya pemeriksaan *stuck* perlu diperhatikan bersama proses pencariannya.

State akhir dan terbaik sebaiknya tetap disajikan terpisah. Setiap konfigurasi dan *run* juga perlu memiliki folder keluaran tersendiri agar hasil tidak tertimpa dan analisis dapat ditelusuri ke data aslinya.

BAB 4
Pembagian Tugas Anggota

Anggota	Pekerjaan utama
Nashiruddin Akram 13524090	Representasi data dan validasi susunan paket, implementasi empat varian Hill-Climbing, dokumentasi penggunaan, dan pengerjaan laporan.
Reinsen Silitonga 13524093	Implementasi Genetic Algorithm, format hasil pencarian, visualisasi, fungsi objektif bonus ETA, pencatatan hasil eksperimen, dan pengerjaan laporan.
Arghawisesa Dwinanda Arham 13524100	Inisialisasi State acak, operasi dan pembangkitan neighbor, implementasi Simulated Annealing, konfigurasi parameter dan validasi hasil SA, penyempurnaan antarmuka web, dan pengerjaan laporan.

Tabel 4.1: Pembagian tugas anggota kelompok.

Referensi

Asisten Laboratorium Inteligensi Artifisial 2023. *Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027*. Versi 1.4, 24 September 2026.

IF3170 Inteligensi Artifisial. (2026). Beyond Classical Search. IF3170 Inteligensi Artifisial, Semester I Tahun Ajaran 2026/2027, Institut Teknologi Bandung.

Poole, D. L., dan Mackworth, A. K. (2023). *Artificial Intelligence: Foundations of Computational Agents*. Edisi ke-3, Bagian 4.6: Local Search. Cambridge University Press. https://artint.info/3e/html/ArtInt3e.Ch4.S6.html.

Baeldung. (2025). Simulated Annealing Explained. https://www.baeldung.com/cs/simulated-annealing

W3Schools. (t.t.). Python Random seed() Method. https://www.w3schools.com/python/ref_random_seed.asp

Python Software Foundation. *random: Generate pseudo-random numbers*. https://docs.python.org/3/library/random.html. Diakses 19 Oktober 2026.

Tim Pengembang Matplotlib. *Quick start guide*. https://matplotlib.org/stable/users/explain/quick_start.html. Diakses 19 Oktober 2026.

MathWorks. How the Genetic Algorithm Works. Diakses 20 Oktober 2026. https://www.mathworks.com/help/gads/how-the-genetic-algorithm-works.html

W3Schools. (t.t.). Python statistics.stdev() Method. https://www.w3schools.com/python/ref_stat_stdev.asp

Plotly. (t.t.). 3D Mesh Plots in JavaScript. https://plotly.com/javascript/3d-mesh/

DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	: - Nashiruddin Akram/13524090 - Reinsen Silitonga/13524093 - Arghawisesa Dwinanda Arham/13524100
Fakultas/Sekolah	: STEI
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027 Kelompok FC Barcelona

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/~~Tidak~~*	2
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	2
Lainnya, Jelaskan: Penggunaan AI membantu dalam merancang kerangka berpikir dalam pembuatan kode, khususnya untuk tampilan front end yang di mana bukan sebuah inti dari tugas besar ini.		2

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Nashiruddin Akram
13524090



Reinsen Silitonga
13524093



Arghawisesa Dwinanda Arham
13524100